Databaskonstruktion (VG-försök)

Innehållsförteckning

1. Databasimplementation 1

1.1. Avgränsning av implementering 1

1.2. Antaganden 2

1.2.1 Primärnycklar - Identifikation av Barn och Nissar 2

1.2.2 Kodifiering av olydighetsnivå och formatet av trackingnummer 3

1.2.3 Hur kvalitetsvärde av inspelningar skall räknas 5

1.2.4 Fokus i subsystemet. 6

1.2.5 Hantering av multivärda attribut - sysslor och medaljer 7

1.3. Datatyper och dess användning 9

1.3.1 TINYINT - ‘olydigNivA’ och ‘kvalitetsVarde’ 9

1.3.2 ENUM - ‘genomForande’ och ‘Medaljnamn’ 11

1.3.3 DATETIME - ‘sysslaTidDatumFardig’ 12

1.4. Implementering av Constraints 13

1.4.1 Check - Barnens personnummer och NOT NULL - Barnens Födelseår 13

1.4.2 Unique - Tomtenissens unika öknamn 14

1.5. Denormalisering 15

1.5.1 Merge, vertikal och horisontell split 15

1.5.1 Merge, vertikal och horisontell split - Förklaring och diskussion 17

1.5.2 Codes 19

1.6. Indexering 21

1.7. Vyer 23

1.7.1 VY Simplification - 1, 2 och 3 (i kodexempel nedan) 23

1.7.2 Specialization - 1 (VG) 25

1.8. Stored procedures 26

1.8.1 Procedur 1 26

1.8.2 Procedur 2 27

1.8.3 Procedur 3 28

1.8.4 Procedur 4 30

1.9. Triggers 32

1.9.1 Trigger 1 - after delete (g) 32

1.9.2 Trigger 2 (g) 34

1.9.3 Trigger 3 (vg). 36

1.9.4 Trigger 4 (vg) 39

1.10. Rights 42

1.10.1 Tabell - ‘ejLevererad_onskeListRader’ 43

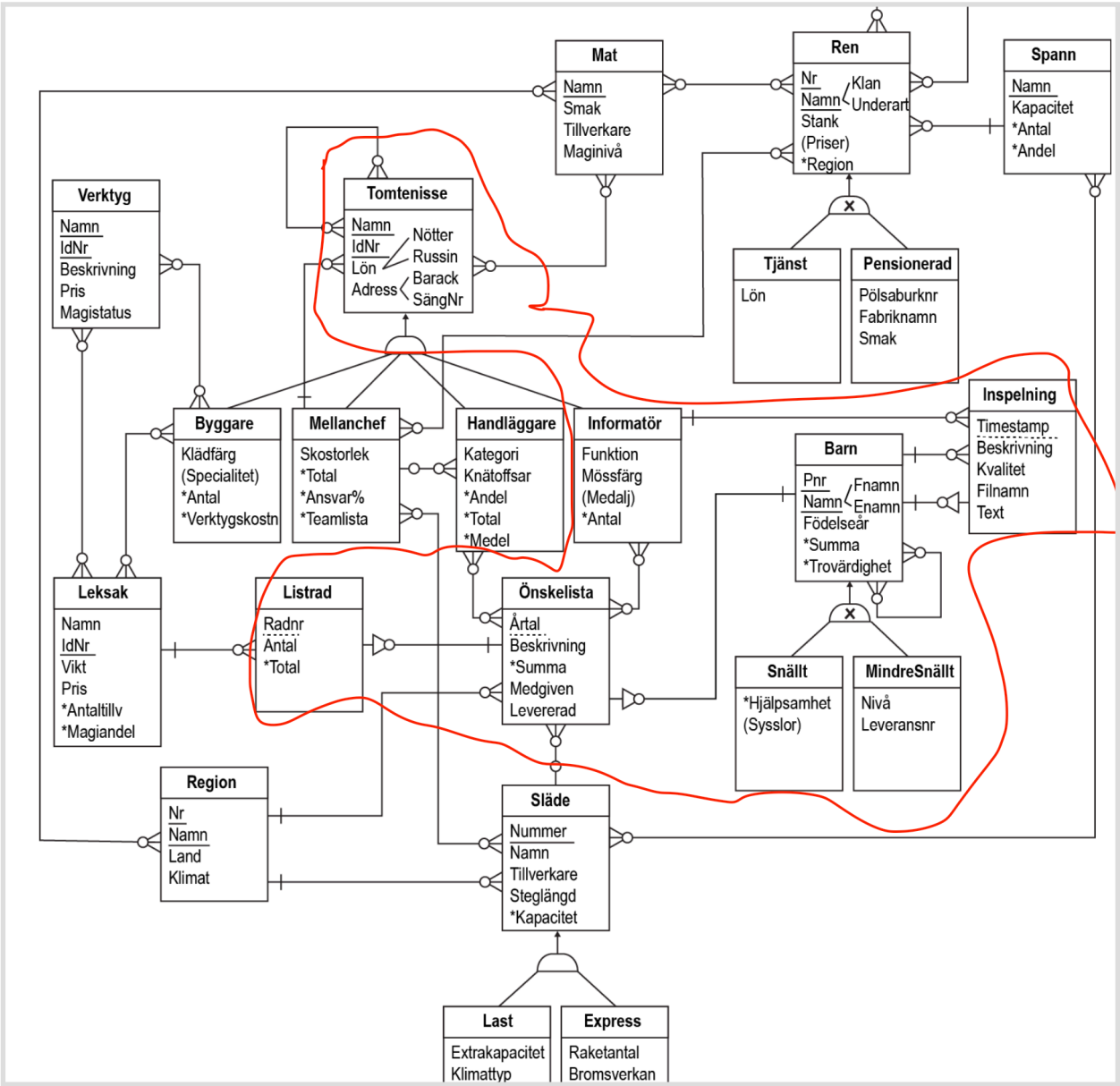
1.10.2 Vy - barnVanner 43

1.10.3 Procedur - hittaInspelningar 44

Referenser 45

1. Databasimplementation

1.1. Avgränsning av implementering



FIGUR 1

I figur 1 går noteras de tabeller vilket inkluderades inom det slutna/avgränsade systemet. Dessa tabeller utgjordes av Barn samt dess arv av ‘snäll och mindre snälla barn’, inspelning, informatör, önskelista och listrad. Av de tabeller/entiteter vilket inkluderades var utgångspunkten utifrån barnets perspektiv och de kompletterande tabellerna valdes med avsikten att utesluta produktionslinjen från systemet och bara fokusera på ett antal use case av ‘barn’ och, indirekt, informatörnissen roll inom gentemot inspelning, men inte till informatörens roll i önskelistan, då relationen beskrivs enligt

följande: ‘Informatören ska vara ansvarig för noll eller flera önskelistor’(Gustavsson & Bjurén, 2022) därmed görs antas inte relationen vara relevant för subsystemet för ‘barnets’ use case och mer riktat åt produktionslinjen.

1.2. Antaganden

1.2.1 Primärnycklar - Identifikation av Barn och Nissar

Inom entiteten ‘barn’ uppger uppgiftsbeskrivningen följande: ‘Barnen registrerar sig själva med sitt födelseår samt sitt unika namn som består av förnamn och efternamn samt barnets unika barn-pnr, vilket utgörs av fyrsiffrigt löpnummer följt av en 6-siffrig regionskod, och slutligen en fyrsiffrig datum-kod (0520 för maj 2020).’ (Gustavsson & Bjurén, 2022). Med detta gjordes antagandet att implementera följande tabellstrukturen (Kodexempel 1):

Kodexempel 1

```
CREATE TABLE barn (
    Pnr char(14),
    Fnamn VARCHAR(20),
    Enamn VARCHAR(20),

     FodelseAR year NOT NULL, -- anges i beskrivningen att det krävs för registrering.
Namnet är inte alltid unikt ensamt. kommer sökas på för att avgöra att det är under 17
och över 1.

    PRIMARY KEY (Pnr, Fnamn, Enamn),
     CHECK (Pnr RLIKE '^[0-9]{4}[0-9]{6}[0-9]{4}$') -- kollar format på barnPnr enl
uppgiftsbeskrivningen

) ENGINE=InnoDB;
```

Inom ‘kodexempel 1’ och ‘Figur 1' identifieras barnet utifrån beskrivningen från Gustavsson och Bjurén (2022), därmed består primärnyckeln av en komposit primärnyckel utgjord av barnets personnummer enligt önskat format, samt den sammansatta nyckeln av för och efternamn.

Implementering av entiteten ‘Tomtenisse’ präglades av Gustavsson och Bjurén (2022), då de angav följande: ‘Tomtenissen identifieras antingen av nissens unika öknamn eller tomte-id. Då tomtenissarna är många så består ett tomte-id av en 6-siffrig regionskod, en fyrsiffrig datum-kod (0520 för maj 2020), en 1-siffrig mösstorlek samt en 9-siffrigt löpnummer med minustecken eller slash-tecken mellan varje del.’ (Gustavsson & Bjurén, 2022). Därmed antogs tabellstrukturen av ‘Tomtenisse’ inom ‘Kodexempel 2’.

Kodexempel 2

```
CREATE TABLE tomteNisse  (
    Namn VARCHAR(20) NOT NULL UNIQUE, -- Constraint: så att namnet är unikt enligt uppg
beskrivningen.
    ID CHAR(23),

    russin int,
    notter int,

     barackNr tinyint, -- valdes för prestanda och för att det rimligtvis är max 255st
baracker
      sangNr smallint, -- vet ej hur mycket en barrack rymmer, men för att akkomodera
arbetare för hela världens juklappar kommer small int väljas.
    PRIMARY KEY (ID),

     CHECK (ID REGEXP '^[0-9]{6}[-/][0-9]{4}[-/][0-9]{1}[-/][0-9]{9}$') -- kollarformat
enl uppgiftsbeskrivningen

) ENGINE=InnoDB;
```

Primärnyckeln för ‘Tomtenisse’ är ID, vilket överensstämmer med ‘antingen av nissens unika öknamn eller tomte-id’ (Gustavsson & Bjurén, 2022). I detta fallet valdes ‘ID’.

Då tomtens öknamn fortfarande skulle vara unikt, implementerades constrainten av NOT NULL UNIQUE för att göra öknamn-kolumnen unik och aldrig tom. Vilket resulterar i att ID agerar primärindex för att identifiera en specifik tuppel i Tomtenisse. Gustavsson och Bjurén (2022) aldrig uppgav att en Tomtenisse ibland inte kunde ha ett öknamn, därmed gjordes antagandet att öknamn är en nödvändighet för varje Tomtenisse och alltid närvarande. Vilket blir fördelaktigt då det delvis speglar verkligheten på denna arbetsplats och kommer kunna möjliggöra unik identifiering med ‘relevant’ information om diverse tomtenissar.

1.2.2 Kodifiering av olydighetsnivå och formatet av trackingnummer

Kodexempel 3

```
-- kodeifierad tabell nedan.
CREATE TABLE olydighetSkala (
    skala tinyint
     CHECK (skala BETWEEN 0 AND 114), -- antagande kommer göras att rött är olydigt och
därmed är 114 max olydighet.
    fargskala varchar (14),
    PRIMARY KEY (skala)
) ENGINE=InnoDB;

_ ————————————————————————————


CREATE TABLE MindreSnalltBarn (
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),

     olydigNivA tinyint , --  kodifieras enligt beskrivning och lagras som en färg (röd
eller grön) i separat tabell.
     trackingNr VARCHAR(20), -- trackingnr antas ha formatet av alfanr. Fickvarchar 20
då mängden julklappar ökar och då måste                                          de vara
flexibla och kunna öka.

    PRIMARY KEY (barnPnr, barnFnamn, barnEnamn),
    FOREIGN KEY (barnPnr, barnFnamn, barnEnamn) REFERENCES barn (Pnr, Fnamn, Enamn),
    FOREIGN KEY (olydigNivA) REFERENCES olydighetSkala (skala)
) ENGINE=InnoDB;
```

```
— ————————————————————————————————

INSERT INTO olydighetSkala (skala, fargskala) VALUES
(0, 'grön'), (1, 'grön'), (2, 'grön'), (3, 'grön'), (4, 'grön'), (5, 'grön'),
(6, 'grön'), (7, 'grön'), (8, 'grön'), (9, 'grön'), (10, 'grön'), (11, 'grön'),
(12, 'grön'), (13, 'grön'), (14, 'grön'), (15, 'grön'), (16, 'grön'), (17, 'grön'),
(18, 'grön'), (19, 'grön'), (20, 'grön'), (21, 'grön'), (22, 'grön'), (23, 'grön'),
(24, 'grön'), (25, 'grön'), (26, 'grön'), (27, 'grön'), (28, 'grön'), (29, 'grön'),
(30, 'grön'), (31, 'grön'), (32, 'grön'), (33, 'grön'), (34, 'grön'), (35, 'grön'),
(36, 'grön'), (37, 'grön'), (38, 'grön'),
(39, 'gul'), (40, 'gul'), (41, 'gul'), (42, 'gul'), (43, 'gul'), (44, 'gul'),
(45, 'gul'), (46, 'gul'), (47, 'gul'), (48, 'gul'), (49, 'gul'), (50, 'gul'),
(51, 'gul'), (52, 'gul'), (53, 'gul'), (54, 'gul'), (55, 'gul'), (56, 'gul'),
(57, 'gul'), (58, 'gul'), (59, 'gul'), (60, 'gul'), (61, 'gul'), (62, 'gul'),
(63, 'gul'), (64, 'gul'), (65, 'gul'), (66, 'gul'), (67, 'gul'), (68, 'gul'),
(69, 'gul'), (70, 'gul'), (71, 'gul'), (72, 'gul'), (73, 'gul'), (74, 'gul'),
(75, 'gul'), (76, 'gul'),
(77, 'röd'), (78, 'röd'), (79, 'röd'), (80, 'röd'), (81, 'röd'), (82, 'röd'),
(83, 'röd'), (84, 'röd'), (85, 'röd'), (86, 'röd'), (87, 'röd'), (88, 'röd'),
(89, 'röd'), (90, 'röd'), (91, 'röd'), (92, 'röd'), (93, 'röd'), (94, 'röd'),
(95, 'röd'), (96, 'röd'), (97, 'röd'), (98, 'röd'), (99, 'röd'), (100, 'röd'),
(101, 'röd'), (102, 'röd'), (103, 'röd'), (104, 'röd'), (105, 'röd'), (106, 'röd'),
(107, 'röd'), (108, 'röd'), (109, 'röd'), (110, 'röd'), (111, 'röd'), (112, 'röd'),
(113, 'röd'), (114, ‘röd');
```

Enligt uppgiftsbeskrivningen från Gustavsson och Bjurén (2022) representeras barnets ‘olydighetsnivå’ i en färgskala, lagrad i ett numeriskt värde mellan 0 och 114. Där representeras grönt de lägre värden och röd representerar de mer olydiga värdena. Denna skala skall användas för identifiering av barnets grad av olydnad. Inom kodexempel 3 definieras ‘olydigNivA’ för ‘MindreSnalltBarn' likt en främmande nyckel till tabellen olydighetsskala, där varje värde mellan 0-114 motsvarar grön, röd och även gul, för representation av barn vilket ligger ‘mittemellan’ röd och grön i spektrumet. Observera hur den gula färgen markerar övergången mellan snäll och mindre snäll.

För utformningen av trackingnummer gav inte Gustavsson och Bjurén (2022) direktiv på hur det skulle formateras. Därmed valdes datatypen av en varchar 20 för att lagra alfanumeriska koder, vilket är en mindre prestanda effektivt lagringsmetod än int typen eller char typen, men fördelen blir en ökad flexibilitet och hantering av fler koljulklappar i framtiden, ifall antalet mindre snälla barn skulle öka.

1.2.3 Hur kvalitetsvärde av inspelningar skall räknas

Kodexempel 4

```
CREATE TABLE inspelning (
    tidsstampel TIMESTAMP,

     barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),
    NisseID CHAR(23),

    barn_om_Pnr CHAR(14), -- barnet som inspelningen handlar om
    barn_om_Fnamn VARCHAR(20),
    barn_om_Enamn VARCHAR(20),

    beskrivning VARCHAR(20),
    kvalitetsVarde tinyint, -- ska ingå i en uträkning
    filNamn VARCHAR (30), -- kanske uttöka pga av meningsstorlek, kan vara längre.
    beskrivandeText VARCHAR(50),

    PRIMARY KEY (tidsstampel, barnPnr, barnFnamn, barnEnamn, NisseID),

    FOREIGN KEY (barnPnr, barnFnamn, barnEnamn) REFERENCES barn (Pnr, Fnamn, Enamn), --
koppling till barn
    FOREIGN KEY (barn_om_Pnr, barn_om_Fnamn, barn_om_Enamn) REFERENCES barn (Pnr,
Fnamn, Enamn), -- koppling till barnet som är subjektet i inspelningen
    FOREIGN KEY (NisseID) REFERENCES InformatorNisse (NisseID), -- direkt koppling till
informator

    CHECK (kvalitetsVarde BETWEEN 1 AND 10) -- kollar så att kvalitetsvärdet är i
spannet av 1-10, viktigt för indexeringen.

) ENGINE=InnoDB;

_ ____________________________

create index inspelningsKvalitet on inspelning(kvalitetsVarde asc) using btree;
```

Inom kodexempel 4 visas implementeringen av tabellen ‘inspelning’, där varje inspelning får ett ‘kvalitetsVarde’. Detta kvalitetsvärde representerar kvaliteten på en inspelning och kommer användas i framtida beräkningar och analyser, förklarat av Gustavsson och Bjurén (2022) skall den exempelvis ingå i uppskattning av ‘trovärdigheten på respektive barn av typen “snällt barn” genom att beräkna ett medelvärde på kvaliteten på alla barnets inspelningar plus antalet genomförda sysslor om barnet hör till typen snälla barn.’.

Gustavsson och Bjurén (2022) angav dock inte vilket format kvalitetsvärdet skulle lagras och därmed ansågs ett numeriskt värde (tinyint) passande, då värdet kommer ingå i matematiska uträkningar. Ytterligare valdes spannet av 1-10 vilket slutligen indexerades för snabbare hämtning, det även i syftet av de matematiska uträkningar värdet kommer ingå i.

1.2.4 Fokus i subsystemet.

Det skapade systemet är utformat utifrån perspektivet av ‘barn’ och entiteter har valts ut för att främja dess funktioner och för maximal exkludering utav större delar av produktionslinjen. I framtagandet av lösningar för uppgiften, skapades främst lösningar för de snälla barnen. Barnets funktioner i systemet inkluderar deltagande i sysslor, skapandet av inspelningar, vänskapsrelationer och framtagandet av önskelistrader/önskelistor. Barnen kategoriseras baserat på beteende, antingen är de "snälla barn" eller "mindre snälla barn” och deras deltagande påverkas av ålders- och beteendekrav. Avgränsningen tillåter även inkluderande av de värden vilket utgör de barnrelaterade-uträkningar. Exempelvis hjälpsamhetsvärde.

Notera exkluderingen av entiteter såsom Leksak, släde och eller region i ‘Figur 1’, då de entiteterna har en indirekt påverkan på hur barn interagerar med systemet. Istället agerar önskelistan likt en barriär/abstraktion av relationen mellan barnet och produktionslinjen då båda subsystem direkt interagerar med önskelistan, men inte direkt med varandra. Därav minimeras komplexitet och fokuset kan läggas på framtagandet av ett passande system för barnen.

1.2.5 Hantering av multivärda attribut - sysslor och medaljer

Inom subsystemet introducerades de multivärda attributen ‘Sysslor’ för de snälla barnen och ‘Medalj’ för informatörnissen. Då dessa är multivärda skapas en många till många relation mellan ursprungstabellen/entiet och det multivärda attributet. Implementeringen går att notera inom ‘Kodexempel 5’ nedan.

Kodexempel 5

```
CREATE TABLE InformatorNisse (
    NisseID CHAR(23),
    Funktion VARCHAR(30), -- Datatyp för att beskriva rollen
    MossFarg VARCHAR(15), -- Datatyp för att beskriva färgen på mössan, lägg constraint
för färgen röd.

    PRIMARY KEY (NisseID), -- Namn och ID som primärnyckel
    FOREIGN KEY (NisseID) REFERENCES tomteNisse (ID),
    CHECK (MossFarg NOT RLIKE 'röd|crimson|merlot')

) ENGINE=InnoDB;


CREATE TABLE Medalj (
    NisseID CHAR(23),
      serieNr char(4), -- implementerade alfanumeriskt nummer för att kunna ha flera
unika loggar av medaljer
    Medaljnamn ENUM('Guldmössan', 'Blå knappen', 'Det Gyldene Russinet'),
    PRIMARY KEY (NisseID, serieNr),
    FOREIGN KEY (NisseID) REFERENCES InformatorNisse (NisseID)
) ENGINE=InnoDB;

_ ___________________________________


CREATE TABLE snalltBarn (
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),

    PRIMARY KEY (barnPnr, barnFnamn, barnEnamn),
    FOREIGN KEY (barnPnr, barnFnamn, barnEnamn) REFERENCES barn (Pnr, Fnamn, Enamn)
) ENGINE=InnoDB;


CREATE TABLE sysslorLogg (
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),

    syssla VARCHAR(20),
    sysslaTidDatumFardig datetime,

    genomForande ENUM('Undermåligt', 'Klåparmässigt', 'Klantigt', 'Lyckat'), -- lades
till enligt beskrivning

       PRIMARY KEY (barnPnr, barnFnamn, barnEnamn, syssla, sysslaTidDatumFardig), --
syssla implementerades i sysslorlogg tabellen, för att undvika duplikat.
        FOREIGN KEY (barnPnr, barnFnamn, barnEnamn) REFERENCES snalltBarn (barnPnr,
barnFnamn, barnEnamn)
) ENGINE=InnoDB;
```

För medaljer beskrevs följande ‘För varje Informatör lagras en lista med namnet på de medaljer (exv “Guldmössan”, “Blå knappen” “Det Gyldene Russinet”) som informatörs-nissen har fått’ (Gustavsson & Bjurén, 2022). Därmed behövde ‘medalj’- tabellen kunna logga flera medaljer per Informatörnisse, därmed implementerades en kompositnyckel.

Först inkluderades en främmande nyckel referens till ‘InformatorNisse’ ID-kolumn i syfte att knyta medaljen till en specifik informatörnisse. Därefter behövde även medaljen kunna identifieras och därmed tupplerna Serienummer ‘serieNr’ i kombination med ‘medaljnamn’. Resulterande blir ett effektivt sätt för loggning av flera spårbara medaljer per informatör. En ytterliggare implementering kan vara inkluderandet av datum för ytterligare spårbarhet, men för subsystemets fokus är det ej nödvändigt.

1.3. Datatyper och dess användning

1.3.1 TINYINT - ‘olydigNivA’ och ‘kvalitetsVarde’

Kodexempel 6

```
CREATE TABLE MindreSnalltBarn (
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),

     olydigNivA TINYINT , --  kodifieras enligt beskrivning och lagras som en färg (röd
eller grön) i separat tabell.
     trackingNr VARCHAR(20), -- trackingnr antas ha formatet av alfanr. Fickvarchar 20
då mängden julklappar ökar och då måste trackingnummrena vara flexibla och kunna öka.

    PRIMARY KEY (barnPnr, barnFnamn, barnEnamn),
    FOREIGN KEY (barnPnr, barnFnamn, barnEnamn) REFERENCES barn (Pnr, Fnamn, Enamn),
    FOREIGN KEY (olydigNivA) REFERENCES olydighetSkala (skala)
) ENGINE=InnoDB;

_ ____________________________

CREATE TABLE inspelning (
    tidsstampel TIMESTAMP,

     barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),
    NisseID CHAR(23),

    barn_om_Pnr CHAR(14), -- barnet som inspelningen handlar om
    barn_om_Fnamn VARCHAR(20),
    barn_om_Enamn VARCHAR(20),

    beskrivning VARCHAR(20),
    kvalitetsVarde TINYINT, -- ska ingå i en uträkning
    filNamn VARCHAR (30), -- kanske uttöka pga av meningsstorlek, kan vara längre.
    beskrivandeText VARCHAR(50),

    PRIMARY KEY (tidsstampel, barnPnr, barnFnamn, barnEnamn, NisseID),

     FOREIGN KEY (barnPnr, barnFnamn, barnEnamn) REFERENCES barn (Pnr, Fnamn, Enamn), --
koppling till barn
     FOREIGN KEY (barn_om_Pnr, barn_om_Fnamn, barn_om_Enamn) REFERENCES barn (Pnr,
Fnamn, Enamn), -- koppling till barnet som är subjektet i inspelningen
     FOREIGN KEY (NisseID) REFERENCES InformatorNisse (NisseID), -- direkt koppling till
informator

     CHECK (kvalitetsVarde BETWEEN 1 AND 10) -- kollar så att kvalitetsvärdet är i
spannet av 1-10, viktigt för indexeringen.

) ENGINE=InnoDB;
```

Inom tabellerna i kodexempel 6 noteras implementeringen av datatypen ‘TINYINT’ (se dem svartmarkerade orden).

I fallet av tabellen ‘MindreSnalltBarn’ lagras olydignivån i en TINYINT, tabellen har i sin tur en kodifierad tabell (‘olydighetSkala’) med en constraint som säkerställer att nivån mellan 0-114. Den implementerade och signerade TINYINTen stödjer tal från -128 till 127, vilket är inom spannet för olydighetskalan. Skulle detta behöva utökas kan int signeras (med spannet 0-255). Tinyint är även passande i mån av prestanda och lagring, då datatypen endast använder 1 byte per lagrat värde och dessutom kan datatypen snabbare användas i numeriska beräkningar och jämförelser. Tillskillnad från exempelvis karaktärer (char).

I fallet av tabellen ‘inspelning’ lagras även ‘kvalitetsVarde’ inom en TINYINT, detta supplementäras av en check constraint vilket specificerar att talet skall ligga i omfånget av 1-10. Detta då ett graderingsystem för inspelningskvalitet ej specificerades av Gustavsson och Bjurén (2022). Här avgör i huvudsak den lilla storleken av det lagrade värdet och dess icke alfanumeriska format. Genom användning av tinyint mängden allokerat minne jämfört med större interger typer, likt exempelvis INT där TINYINT sparar in hela 7 bitar .

1.3.2 ENUM - ‘genomForande’ och ‘Medaljnamn’

Kodexempel 7

```
CREATE TABLE Medalj (
    NisseID CHAR(23),
      serieNr char(4), -- implementerade alfanumeriskt nummer för att kunna ha flera
unika loggar av medaljer
    Medaljnamn ENUM('Guldmössan', 'Blå knappen', 'Det Gyldene Russinet'),
    PRIMARY KEY (NisseID, serieNr),
     FOREIGN KEY (NisseID) REFERENCES InformatorNisse (NisseID)
) ENGINE=InnoDB;

_ ______________________

CREATE TABLE sysslorLogg (
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),

     syssla VARCHAR(20),
     sysslaTidDatumFardig datetime,

      genomForande ENUM('Undermåligt', 'Klåparmässigt', 'Klantigt', 'Lyckat'), -- lades
till enligt beskrivning

       PRIMARY KEY (barnPnr, barnFnamn, barnEnamn, syssla, sysslaTidDatumFardig), --
syssla implementerades i sysslorlogg tabellen, för att undvika duplikat.
        FOREIGN KEY (barnPnr, barnFnamn, barnEnamn) REFERENCES snalltBarn (barnPnr,
barnFnamn, barnEnamn)
) ENGINE=InnoDB;
```

Inom ‘Kodexempel 7’ noteras hur genomförande och medaljnamn tillgivits datatypen av ENUM för identifierande av hur en syssla genomförts, samt för identifiering av informatörnissarnas medaljer. Fördelen med datatypen är automatisk indexering av värdena, vilket underlättar filtrering och sökning.

Anledningen till valet av datatypen enum för medaljnamn, grundade sig i följande påstående: ‘För varje informatör lagras en lista med namnet på de medaljer (exv “Guldmössan”, “Blå knappen” “Det Gyldene Russinet”) som informatörs-nissen har fått’ (Gustavsson & Bjurén, 2022). Därmed erhölls tre fördefinerade exempelfraser av medaljtyper vilket implementerades in i enum-attributet och ingen utökning av fler medaljtyper gjordes, då det inte gjorde nytta för den centrala användaren, barnet.

För tabellen sysslorLogg tillhandahålls skalan på hur en syssla genomförs enligt följande text ‘För de snälla barnen ska en beskrivning av vilka sysslor barnen genomfört och till vilken grad barnen fullgör sysslorna, beskrivet av nivåerna “Undermåligt”, “Klåparmässigt”, “Klantigt” och “Lyckat”’ (Gustavsson & Bjurén, 2022). Därmed implementerades skalan med de nämnda, fördefinerade fraserna.

1.3.3 DATETIME - ‘sysslaTidDatumFardig’

Kodexempel 8

```
CREATE TABLE sysslorLogg (
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),

     syssla VARCHAR(20),
     sysslaTidDatumFardig datetime,

      genomForande ENUM('Undermåligt', 'Klåparmässigt', 'Klantigt', 'Lyckat'), -- lades
till enligt beskrivning

       PRIMARY KEY (barnPnr, barnFnamn, barnEnamn, syssla, sysslaTidDatumFardig), --
syssla implementerades i sysslorlogg tabellen, för att undvika duplikat.
        FOREIGN KEY (barnPnr, barnFnamn, barnEnamn) REFERENCES snalltBarn (barnPnr,
barnFnamn, barnEnamn)
) ENGINE=InnoDB;
```

Enligt Gustavsson och Bjurén (2022) implementerades sysslor i anknytning till entiteten av ‘snälla barn’. I samband med detta gjordes antagandet angående hur barnen utför dem, i detta fallet antogs att de loggar varje syssla själva och för att bibehålla varje sysslas unikhet, dvs de ska kunna göra samma syssla flera gånger, implementerades ‘sysslaTidDatumFardig’ till datatypen: ‘DATETIME’. Datatypen tillåter precis lagring av både datum och tid i ett fördefinerat format, vilket effektivt kan användas i frågespråk för filtrering, sortering, gruppering av tidsspann och det ger de snälla barnen möjligheten till att ingå i multipla poster av samma syssla under samma dag. Sysslorna blir mer unika och speglar då verkligheten och användarens ändamål bättre.

1.4. Implementering av Constraints

1.4.1 Check - Barnens personnummer och NOT NULL - Barnens Födelseår

Kodexempel 9

```
CREATE TABLE barn (
    Pnr char(14),
    Fnamn VARCHAR(20),
    Enamn VARCHAR(20),

     FodelseAR year NOT NULL, -- anges i beskrivningen att det krävs för registrering. Namnet är inte alltid unikt ensamt. kommer sökas på för att avgöra att det är under 17 och över 1.

    PRIMARY KEY (Pnr, Fnamn, Enamn),
     CHECK (Pnr RLIKE '^[0-9]{4}[0-9]{6}[0-9]{4}$') -- kollar format på barnPnr enl uppgiftsbeskrivningen

) ENGINE=InnoDB;
```

Inom ‘Kodexempel 9’ implementerades en ‘NOT NULL’ constraint på variabeln av födelseår, detta gjordes då ‘Barnen registrerar sig själva med sitt födelseår samt sitt unika namn som består av förnamn och efternamn samt barnets unika barn-pnr’ (Gustavsson & Bjurén, 2022). Tolkningsvis ska då, enligt Gustavsson och Bjurén (2022), födelseåret av barnen inkluderas då det är en del av registreringsprocessen av ett barn, mot subsystemet. Implementeringen av ‘NOT NULL’ säkerställer bidrar därmed till datakonsekvens gentemot uppgiftens beskrivning. Födelseåret identifierar även ifall barnet är i rätt åldersomfång för att kunna önska julklappar.

Inom ‘Kodexempel 9’ implementerades även en ‘CHECK’ constrainten på barnens personnummer, ‘Pnr’ för korrekt formatering av barnets personnummer ‘vilket utgörs av fyrsiffrigt löpnummer följt av en 6-siffrig regionskod, och slutligen en fyrsiffrig datum-kod (0520 för maj 2020)’ (Gustavsson & Bjurén, 2022).

1.4.2 Unique - Tomtenissens unika öknamn

Kodexempel 10

```
CREATE TABLE tomteNisse  (
    Namn VARCHAR(20) NOT NULL UNIQUE, -- Constraint: så att namnet är unikt enligt uppg
beskrivningen.
    ID CHAR(23),

    russin int,
    notter int,

    barackNr tinyint, -- valdes för prestanda och för att det rimligtvis är max 255st
baracker
    sangNr smallint, -- vet ej hur mycket en barrack rymmer, men för att akkomodera
arbetare för hela världens juklappar kommer small int väljas.
    PRIMARY KEY (ID),

    CHECK (ID REGEXP '^[0-9]{6}[-/][0-9]{4}[-/][0-9]{1}[-/][0-9]{9}$') -- kollarformat
enl uppgiftsbeskrivningen

) ENGINE=InnoDB;
```

Inom ‘Kodexempel 10’ implementerades en ‘UNIQUE’ constraint i kombination med en ‘NOT NULL’ på attributet ‘Namn’. Detta gjordes i förhållande till beskrivningen: ‘Tomtenissen identifieras antingen av nissens unika öknamn eller tomte-id’ (Gustavsson & Bjurén, 2022). I subsystemet utsågs Nissens personliga ‘ID’ likt primärnyckel, men för en tydlig spegling av beskrivningen implementerades ‘UNIQUE’ constraint för bibehållandet av ‘unika öknamn’ (Gustavsson & Bjurén, 2022). ‘NOT NULL’ implementerades då nissarnas öknamn antogs vara nödvändiga utifrån en tolkning av Gustavsson och Bjuréns (2022) beskrivning. Öknamnet kan exempelvis antas vara nödvändigt, likt barnen namn, för registrering in till subsystemet.

1.5. Denormalisering

1.5.1 Merge, vertikal och horisontell split

Kodexempel 11

Steg 1 Ursprungstabeller – Innan merge (och split!!!!!)

```
CREATE TABLE onskelista (
    arTal YEAR,
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),
         leveransStatus  ENUM('levererad',  'ej  levererad'),  – exkluderades  efter  den
vertikala splitten
    PRIMARY KEY (barnPnr, barnFnamn, barnEnamn, arTal),
    FOREIGN KEY (barnPnr, barnFnamn, barnEnamn)
        REFERENCES barn (Pnr, Fnamn, Enamn)
) ENGINE=InnoDB;


CREATE TABLE listrad (
    arTal YEAR,
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),
    radNr TINYINT,
    antal TINYINT,
    beskrivning VARCHAR(50),
    Kommentar varchar (50),
    PRIMARY KEY (arTal, barnPnr, barnFnamn, barnEnamn, radNr),
    FOREIGN KEY (barnPnr, barnFnamn, barnEnamn, arTal)
        REFERENCES onskelista (barnPnr, barnFnamn, barnEnamn, arTal)
) ENGINE=InnoDB;
```

Steg 2 – Vertikal split på en mergad tabell av önskelista och listrad.

```
CREATE TABLE ejLevererad_onskeListRader (
    arTal year,
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),
    -- medGivande = irrelevant då handläggare avgränsas
    radNr tinyint,
    antal tinyint,

     PRIMARY KEY (arTal, barnPnr, barnFnamn, barnEnamn , radNr),
     FOREIGN KEY ( barnPnr, barnFnamn, barnEnamn) REFERENCES barn (Pnr, Fnamn, Enamn)

) ENGINE=InnoDB;

_ ____________________
CREATE TABLE levererade_onskeListRader (
    arTal year,
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),
    radNr tinyint,
    antal tinyint,

     PRIMARY KEY (arTal, barnPnr, barnFnamn, barnEnamn , radNr),
     FOREIGN KEY ( barnPnr, barnFnamn, barnEnamn) REFERENCES barn (Pnr, Fnamn, Enamn)

) ENGINE=InnoDB;

_ ________
```

Steg 3 – Horisontell split på kommentarer och beskrivningar

-- Undviker att kommentarer lagras för varje kolumn.

```
-- enligt uppgiftsbeskrivningen lagras en kommentar 'då och då' för varje önskelistrad.

CREATE TABLE listRad_kommentar (
    arTal year,
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),
    radNr tinyint,

    Kommentar varchar (50),
    -- "Mindre färgskiftning i målning, Otydlig markering.”

     beskrivning varchar (50),
    -- barnens beskrivning!

     PRIMARY KEY (arTal, barnPnr, barnFnamn, barnEnamn , radNr)
) ENGINE=InnoDB;
```

1.5.1 Merge, vertikal och horisontell split - Förklaring och diskussion

Steg 1 Merge: Inom ‘Kodexempel 11’ noteras implementering av merge på tabellerna under ‘Urprungstabeller’. Merge görs på dessa tabeller då de, inom ‘Figur 1’, inkluderas i en 1 till M relation där poster i listrad-tabellen är beroende av att en motsvarande rad existerar inom önskelist-tabellen. ‘Varje önskelista har noll eller flera rader, och en rad identifieras av den önskelista som raden är kopplad till tillsammans med radens listnummer.’ (Gustavsson & Bjurén, 2022). Tabellerna är därmed starkt beroende av varandra och en merge hade gjort det lättare att komma åt en maximal mängd relaterad data per tuppel.

Strukturen ‘önskelista’ och ‘listrad’ (Kodexempel 11) kan jämföras dem med strukturen av ‘levererade_onskeListRader’ (tabellen är vertikalt splittrad och präglas av mergen). ‘levererade_onskeListRader’ struktur är ett resultat av att ha flyttat de unika attributen från ‘önskelista’, till ‘listrad’ då den redan hade mestadels av den gemensamma kompositnyckeln. Foreign key till önskelista-tabellen togs bort då den nu är borttagen och dess unika attribut inkluderas med i ‘listrad’. De främmande nyckelattributen från barn behölls.

Den mergade tabellen hade därmed följande attributen: arTal YEAR, barnPnr CHAR(14), barnFnamn VARCHAR(20), barnEnamn VARCHAR(20), radNr TINYINT, antal TINYINT, beskrivning VARCHAR(50), Kommentar varchar (50) och leveransStatus ENUM('levererad', 'ej levererad’). (Notera att leveransstatus sedan togs bort efter den vertikala splitten).

Användningen av merge gjordes på tabellerna ‘önskelista’ och ‘listrad’ i första hand då dessa tabeller utgörs av en: 1 till m relation. Där poster inom ‘listrad’ identifieras av en kompositnyckel vilket har främmande nycklar från ‘önskelistan‘ kompositnyckel. Alltså har de en likadan nyckelstruktur och ett kontextuellt beroende, vilket understryker implementationen och gör den lättare. Då endast Önskelistans unika enkla attribut behöver flyttas till den svaga tabellen ‘listrad’ och sedan tas referensen till ‘önskelistan’ bort. Redundans tillförs i syftet av en ökad prestanda, då komplexa sökningar, frågespråk och join-satser undviks mellan ‘önskelista’ och ‘listrad’, vilket resulterar i en mer lättarbetad data. Vilket ger en komplett bild av vems önskelistrad det är, vilket år, hur många leksaker per rad och vilket radnummer önskelistrader har. Nackdelarna med denna implementeringen, är en ökad minnesallokering per tuppel, pågrund av redundansen. Men då det är billigare med lagring än datorkraft görs föredras denna merge.

Steg 2 Vertikal split: Därefter utfördes en vertikal split på den mergade önskelistrad-tabellen. Detta gjordes genom en duplicering av den mergade tabellen av ‘önskelistrad’ och syftet med splitten var att representera två olika värden av leveransstatus: levererad och ej levererad. Därmed togs attributet ‘leveransStatus’ bort och inkluderades inte explicit i de splittrade tabellerna. Tabellernas struktur dvs nycklar och attribut var fortsatt likadana efter splitten och kolumner korresponderar mellan dem.

Främmande nycklar behålls densamma, då de båda har nyckelreferenser från barn och denna relationen förblir giltig för både ‘levererade’ och ‘ej levererade’ önskelistrader. Detta möjliggör att alla barn ska kunna ha flera poster i båda tabellerna och sedan ska posterna i 'ej levererade’ önskelistrader kunna prioriteras och behandlas för att flyttas över till de ‘levererade’ önskelistrader.

Motiverande för denna implementeringen av vertikal split baserades på vilken kolumn jag tror barn och eller informatörer skulle söka på. Barn kanske vill kunna se sin leveransstatus på önskade julklappar, det kan även vara relevant för produktionslinjen och resten av nissarna då de indirekt påverkar leveranskedjan. Genom att då ta bort leveransstatus - attributet minskar komplexiteten för varje tuppel och de skapade tabellerna blir mindre, vilket gör det lättare att utföra sökningar på relevanta leveransstatusar då detta implicit förmedlas via vilken tabell tuppeln existerar inom. En ytterligare spekulering är att ‘ej levererade’ leveranser efterfrågas mer frekvent då produktionslinjen måste följa upp dessa ordrar och se till att leverera dem i efterhand.

Sammanfattningsvis blir produkten av den vertikala splitten tabellerna: ’ejLevererad_onskeListRader’ och ‘levererade_onskeListRader’. Dessa skapas genom att duplicera deras ursprungstabell , i mindre tabeller baserat på ett värde, utan en påverkan på dess ursprungliga tabellstruktur. Vilket minskar datamängden per tabell, tack vare att det nu finns två tabeller istället för en enda tabell och därmed kan posterna fördelas baserat på leveransstatus, och genom exkluderingen av attributet ‘leveransStatus’ vilket gör att mindre data behöver lagras per tuppel. Detta förbättrar, i sin tur, prestanda i frågespråk, filtrering och sökning m.m. Vilket är aktuellt då önskelistor/önskelistrader är en central nod inom systemet, vilket förmodligen kommer efterfrågas mycket.

Steg 3 Horisontell split: Den horisontella splitten görs på attributen/kolumnerna av ‘kommentar’ och ‘beskrivning’. Det görs genom skapandet av en separat tabell ‘listRad_kommentar’, vilket lagrar de inkonsistent förekommande kommentarer och beskrivningarna. Inom ‘listRad_kommentar’ finns hela kompositnyckeln av de vertikalt splittade tabellerna, detta gör den implicit kopplad till både ‘ejLevererad_onskeListRader‘ och ‘levererade_onskeListRader’. Ingen referens görs till en specifik önskelistrad-tabellerna eller båda av dem, då det skulle leda till en felaktig nyckel referens, eller och en ensidig relation till bara en av önskelistrad-tabellerna. Därför blir denna approachen passande, då den horisontella splitten blir giltig för båda av de vertikalt splittade tabellerna: ‘ejLevererad_onskeListRader‘ och ‘levererade_onskeListRader’.

Implementeringen separerar de inkonsistent förekommande och ofta större attributen: ‘kommentar’och ‘beskrivning’, vilket är av datatypen varchar (50), till en ny tabell. Därmed minskas frekvensen av nullvärden och potentiellt upp till ca 200 bytes per attribut, per tuppel inom ‘ejLevererad_onskeListRader‘ och ‘levererade_onskeListRader’ ifall en kommentar av varchar (50) lagras. Detta minskar den genomsnittliga mängden data som lagras i de primära tabellerna, samt förbättrar effektiviteten och prestandan när det gäller datahämtning och bearbetning.

Nackdelarna med denna approachen är att frågespråk blir mer komplicerat då en join måste göras mellan önskelistrad-tabellerna och den horisontellt splittrade ‘listRad_kommentar’ för en komplett databild ifall en kommentar har lagrats för en specifik levererad eller icke levererad önskelistrad. Inom önskelistrad-tabellerna finns heller ingen indikator på att en kommentar har lagrats till dess att en sökning eller join görs till ‘listRad_kommentar’. Då det inte finns en främmande nyckelreferens integritet att upprätthålla, till en specifik tabell, ökas risken för icke överensstämmande poster mellan ‘listRad_kommentar’ till ’ejLevererad_onskeListRader’ och ‘levererade_onskeListRader’. Ett problem vilket potentiellt kan lösas med hjälp av ‘triggers'.

1.5.2 Codes

Kodexempel 12

```
CREATE TABLE olydighetSkala (
    skala tinyint
     CHECK (skala BETWEEN 0 AND 114), -- antagande kommer göras att rött är olydigt och
därmed är 114 max olydighet.
    fargskala varchar (14),
    PRIMARY KEY (skala)
) ENGINE=InnoDB;


CREATE TABLE MindreSnalltBarn (
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),

     olydigNivA tinyint , --  kodifieras enligt beskrivning och lagras som en färg (röd
eller grön) i separat tabell.
     trackingNr VARCHAR(20), -- trackingnr antas ha formatet av alfanr. Fickvarchar 20
då mängden julklappar ökar och då måste trackingnummrena vara flexibla och kunna öka.

    PRIMARY KEY (barnPnr, barnFnamn, barnEnamn),
    FOREIGN KEY (barnPnr, barnFnamn, barnEnamn) REFERENCES barn (Pnr, Fnamn, Enamn),
    FOREIGN KEY (olydigNivA) REFERENCES olydighetSkala (skala)
) ENGINE=InnoDB;


INSERT INTO olydighetSkala (skala, fargskala) VALUES
(0, 'grön'), (1, 'grön'), (2, 'grön'), (3, 'grön'), (4, 'grön'), (5, 'grön'),
(6, 'grön'), (7, 'grön'), (8, 'grön'), (9, 'grön'), (10, 'grön'), (11, 'grön'),
(12, 'grön'), (13, 'grön'), (14, 'grön'), (15, 'grön'), (16, 'grön'), (17, 'grön'),
(18, 'grön'), (19, 'grön'), (20, 'grön'), (21, 'grön'), (22, 'grön'), (23, 'grön'),
(24, 'grön'), (25, 'grön'), (26, 'grön'), (27, 'grön'), (28, 'grön'), (29, 'grön'),
(30, 'grön'), (31, 'grön'), (32, 'grön'), (33, 'grön'), (34, 'grön'), (35, 'grön'),
(36, 'grön'), (37, 'grön'), (38, 'grön'),

(39, 'gul'), (40, 'gul'), (41, 'gul'), (42, 'gul'), (43, 'gul'), (44, 'gul'),
(45, 'gul'), (46, 'gul'), (47, 'gul'), (48, 'gul'), (49, 'gul'), (50, 'gul'),
(51, 'gul'), (52, 'gul'), (53, 'gul'), (54, 'gul'), (55, 'gul'), (56, 'gul'),
(57, 'gul'), (58, 'gul'), (59, 'gul'), (60, 'gul'), (61, 'gul'), (62, 'gul'),
(63, 'gul'), (64, 'gul'), (65, 'gul'), (66, 'gul'), (67, 'gul'), (68, 'gul'),
(69, 'gul'), (70, 'gul'), (71, 'gul'), (72, 'gul'), (73, 'gul'), (74, 'gul'),
(75, 'gul'), (76, 'gul'),

(77, 'röd'), (78, 'röd'), (79, 'röd'), (80, 'röd'), (81, 'röd'), (82, 'röd'),
(83, 'röd'), (84, 'röd'), (85, 'röd'), (86, 'röd'), (87, 'röd'), (88, 'röd'),
(89, 'röd'), (90, 'röd'), (91, 'röd'), (92, 'röd'), (93, 'röd'), (94, 'röd'),
(95, 'röd'), (96, 'röd'), (97, 'röd'), (98, 'röd'), (99, 'röd'), (100, 'röd'),
(101, 'röd'), (102, 'röd'), (103, 'röd'), (104, 'röd'), (105, 'röd'), (106, 'röd'),
(107, 'röd'), (108, 'röd'), (109, 'röd'), (110, 'röd'), (111, 'röd'), (112, 'röd'),
(113, 'röd'), (114, 'röd');
```

För implementeringen av kodifiering (‘Codes’) togs följande direktiv: ‘För de mindre snälla barnen lagras en nivå på hur olydiga barnen varit. Värdet representeras av en färg från grönt, gult till rött, lagrat som ett tal mellan 0-114’ (Gustavsson & Bjurén, 2022).

Den centrala delen av denna lösning är tabellen ‘olydighetSkala’, vilket lagrar färgerna (grön, gul och röd) i relation till en numerisk skala från 0 till 114. Denna skala används sedan som referens i tabellen ‘MindreSnalltBarn’, där varje barns olydighetsnivå lagras som ett numeriskt värde i kolumnen ‘olydigNivA’. Detta möjliggör att färgen för ett barns nivå kan härledas från skalan, istället för att lagra färgkoden direkt i posterna av ‘MindreSnalltBarn. Det ger en effektivisering genom att inte duplicera textsträngar för varje barn.

För att förhålla sig till Gustavsson och Bjuréns (2022) gjordes sedan inserts där varje färg representerades av en färg representerar en siffra på 0-114. Antagandet angående dispositionen av skalan kan noteras inom inserten i 'Kodexempel 12’. Där färgskalan delades i tre kategorier ‘grön’, ‘gul’ och ‘röd’ vilket fördelades jämt över varsin tredjedel av sifferskalan på ca 38 siffror var. Antagandet gjordes över att grön ska representera en mindre nivå, gul måttlig och röd en hög nivå av olydighet. Samt ska siffrorna avgöra vilken nivå man ligger på inom färgskalan, exempelvis är ’40, gul’ mer olydig än ’20, gul’.

Fördelen är att man i detta fallet undviker att lagra ‘fargskala varchar (14)’ för varje post inom ‘MindreSnalltBarn’. Istället lagras en tinyint som refererar till en korresponderande sifferskala och färg, i tabellen ‘olydighetsskala’. Detta sparar i upp till 15 bytes per tuppel för varje färgskala som istället representeras av en tinyint på 1 byte. ‘MindreSnalltBarn’ blir därmed mer effektiv i frågespråk, och sökning, i utbyte mot en lite mer komplicerad frågespråk-sats för framtagandet av den exakta färgtexten inom varje mindre snällt barns tuppel.

Systemet kan även skalas med fler färger och en bredare sifferskala då tinyinten är signerad enligt mysql standard vilket ger den ett spann av -128-127 men om man signerar den i ‘olydighetsskala’ kan ‘skala’ storlek utökas i upp till 255 och därmed fler färger adderas för representation av fler olydighetsnivåer.

1.6. Indexering

Kodexempel 13

```
 create index inspelningsKvalitet on inspelning(kvalitetsVarde asc) using
  btree;
show index from inspelning; -- kollar indexet som skapades

 create index barnAlder on barn ( FodelseAR asc) using btree;
show index from barn; -- kollar indexet som skapades
```

Table	Non_unique	Key_name	Seq_in_i...	Column_name	Collation	Cardinality	Sub_part	Packed	Null
inspelning	0	PRIMARY	1	tidsstampel	A	13	NULL	NULL	
inspelning	0	PRIMARY	2	barnPnr	A	13	NULL	NULL	
inspelning	0	PRIMARY	3	barnFnamn	A	13	NULL	NULL	
inspelning	0	PRIMARY	4	barnEnamn	A	13	NULL	NULL	
inspelning	0	PRIMARY	5	NisseID	A	13	NULL	NULL	
inspelning	1	barnPnr	1	barnPnr	A	10	NULL	NULL	
inspelning	1	barnPnr	2	barnFnamn	A	10	NULL	NULL	
inspelning	1	barnPnr	3	barnEnamn	A	10	NULL	NULL	
inspelning	1	barn_om_Pnr	1	barn_om_Pnr	A	6	NULL	NULL	YES
inspelning	1	barn_om_Pnr	2	barn_om_Fnamn	A	6	NULL	NULL	YES
inspelning	1	barn_om_Pnr	3	barn_om_Enamn	A	6	NULL	NULL	YES
inspelning	1	NisseID	1	NisseID	A	6	NULL	NULL	
inspelning	1	inspelningsKvalitet	1	kvalitetsVarde	A	6	NULL	NULL	YES

FIGUR 2 - INSPELNINGSKVALITET

Table	Non_unique	Key_name	Seq_in_i...	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type
barn	0	PRIMARY	1	Pnr	A	10	NULL	NULL		BTREE
barn	0	PRIMARY	2	Fnamn	A	10	NULL	NULL		BTREE
barn	0	PRIMARY	3	Enamn	A	10	NULL	NULL		BTREE
barn	1	barnAlder	1	FodelseAR	A	10	NULL	NULL		BTREE

Result 32

FIGUR 3 - BARNALDER

Inom ‘kodexempel 13’ noteras implementeringen av index på kolumnerna ‘kvalitetsVarde’ från tabellen ‘inspelning’ och ‘FodelseAR’ från ‘barn’ tabellen. För skapandet av index användes ‘Create index’, följt av indexets deskriptiva namn och sedan vilken tabell respektive kolumn man skapar indexet på. Slutligen används btree, vilket är en dataträdsstruktur.

‘kvalitetsVarde’ valdes då det är centralt i uträkningen för att uppskatta hur ‘trovärdigt’ ett ‘snällt’ barn är. Utifrån följande direktiv:

‘Det ska vara möjligt att uppskatta trovärdigheten på respektive barn av typen “snällt barn” genom att beräkna ett medelvärde på kvaliteten på alla barnets inspelningar plus antalet genomförda sysslor om barnet hör till typen snälla barn’ (Gustavsson & Bjurén, 2022).

Därmed används indexet för att optimera denna uträkning. För ytterligare frågespråk vilket filtrerar outputen baserat på specifika värden av ‘kvalitetsVarde’ kommer indexeringen på värdet agera likt en sorterad lista där man snabbt kommer kunna åtkomma och bearbeta dess värden. Ifall subsystemet skall generera statistik eller rapporter baserat på kvalitet, kommer detta index att underlätta snabbare datahämtning, vilket därmed snabbar upp den kompletta ‘databilden’. Detta gynnar informatörer, då det ger dem snabb åtkomst för bedömningar eller sökning och av ett specifikt eller flera barns inspelningar baserat på deras kvalitetsvärde. Ur barnets perspektiv är det av nytta då de får ‘läsa sina egna önskelistor eller de inspelningar/inlägg som de gjort’(Gustavsson & Bjurén, 2022) och då kan de få tillgång till de dåliga inspelningarna för förbättringar.

Barnen registrerar sig med b.la ‘FodelseAR’ därefter måste följande krav kontrolleras ‘Ett barn får inte skriva en önskelista innan barnet har fötts eller efter året det fyllt 17 år.’ (Gustavsson & Bjurén, 2022). Detta skapar ett behov för att subsystemet snabbt ska kunna sortera och identifiera de barn vilket är påväg in dvs ska lägga sina första önskningar och vilka som är påväg ut dvs lägger eller har lagt sina sista önskningar. Denna indexeringen kommer då i framtiden att exempelvis snabbare validera ålderskriteriet för handläggning av barnens önskelistor och undviker då att 18+ åringar lägger önskningar. Därav exempelvis nissar, med behörighet, kommer få möjligheten att kontrollera denna åldersbegränsning effektivare.

1.7. Vyer

1.7.1 VY Simplification - 1, 2 och 3 (i kodexempel nedan)

Kodexempel 14

```
-- VYER (för barnen)
--  ger  barn  en  överblick  till  alla  önskelistor  och  ger  en  helhetsbild  av  alla
önskelistrader för använding av exmpelvis statistik eller i produktionslinjen.

-- simplification 1

CREATE VIEW allaOnskelistor AS
select * from ejLevererad_onskeListRader
union
select * from levererade_onskeListRader;

select * from allaOnskelistor;

-- Simplification 2, visar sysslor anpassat till barnet och informatören

-- simplification 2
CREATE VIEW BarnSysslorSammanfattning AS
SELECT
  barnFnamn,
  barnEnamn,
  syssla as Syssla,
  sysslaTidDatumFardig as Fardig,
  genomForande as Betyg
FROM
  sysslorLogg;
select * from BarnSysslorSammanfattning;


-- simplification 3 visar vänskaper från ett tvåsidigt perspektiv. Därmed kommer man
kunna söka på pnr1 och matcha det med person a vilket ger ALLA dess relaterade vänner.
CREATE VIEW barnVanner AS
SELECT
    Pnr1 as Personnummer,
    Fnamn1 as Fornamn,
    Enamn1 as EfterNamn,
    Pnr2 as Kompisens_Personnummer,
    Fnamn2 as Kompisens_Fornamn,
    Enamn2 as Kompisens_EfterNamn
FROM
    barn_vanskap
UNION
SELECT
    Pnr2 as Personnummer,
    Fnamn2 as Fornamn,
    Enamn2 as EfterNamn,
    Pnr1 as Kompisens_Personnummer,
    Fnamn1 as Kompisens_Fornamn,
    Enamn1 as Kompisens_EfterNamn
FROM
    barn_vanskap;

select * from barnVanner;
```

I skapandet av en vy görs en virtuell tabell med resultatet utifrån dess inkluderade frågespråk. Den virtuella tabellen existerar inte då den inte lagrar någon data självständigt. Funkar likt en kamera för att visa den databild utifrån den definierade frågan och utan att spara bilden.

Vyn ‘allaOnskelistor’ (‘Kodexempel 14’) gör följande projektioner: ’select * from ejLevererad_onskeListRader’ och ‘select * from levererade_onskeListRader;’ och emellan dem finns uttrycket ‘union’, vilket kombinerar resultaten projektionerna, tar bort dubbletter och returnerar en kombinerad uppsättning av rader. I detta fallet kombineras ej-leverarde och levererade önskelistrader för att få en komplett bild av alla önskelistrader någonsin. Användningen av union är fördelaktig då tabellerna är splittade på värdet av leveransstatus och därmed finns inga dubbletter, samt är kolumnstrukturen identisk i båda tabellerna vilket undviker nullvärden för unika kolumner. Enligt direktiv ska barn ‘få mata in person-information om sig själva i systemet samt läsa sina egna önskelistor’ (Gustavsson & Bjurén, 2022). Denna vyn understödjer påståendet genom att ge barn tillgång till en helhetsbild av önskelistrader oberoende utav leveransstatus.

Inom 'Kodexempel 14, simplification 2’ skapas en vy med namnet ‘BarnSysslorSammanfattning’ vilket har i syfte att ge en komplett bild över vilka sysslor som alla barnen har gjort. Vyn börjar med en projektion från tabellen ‘sysslorLogg’, där inkluderas följande data: det snälla barnets förnamn, efternamn, sysslan utförd, samt när och hur den utfördes. Barnets personnummer exkluderades avsiktligt då det inte alltid är relevant för barnen. Implementeringen gjordes också med avsikten att förhålla sig till följande direktiv ‘En informatör har bara tillgång till en begränsad uppsättning information, bestående av önskelistorna, inspelningarna och information om barnen.’ (Gustavsson & Bjurén, 2022) vilket ej specificerar hur mycket information de skall ha om barnen och därmed görs antagandet att det inte alltid inkluderar barnens personnummer. Denna vyn blir därmed av nytta för barnen och informatörer då de ska ha relevant information om sysslor, med fokus på sysslor.

Inom 'Kodexempel 14, simplification 3’ skapas vyen ‘barnVanner’ med syftet att simplifiera den komplexa M till N förhållandet mellan barn. Då ett vänskapsförhållande är tvåsidigt, dvs kan barn 1 vara vän med barn 2, samtidigt som barn 2 är vän med barn 1. Denna vyn gör då två projektioner av samma attribut från samma tabell ‘barn_vanskap’, för att kunna visa båda sidor av vänskapsförhållandet. Först väljs Pnr1, Fnamn1, och Enamn1 som barnets information och Pnr2, Fnamn2, och Enamn2 representerar kompisens uppgifter. Därefter görs en andra projektion, där samma information om barn och kompis speglas, men med omvänd ordning, dessa två projektioner joinas med hjälp av ‘union’ operatorn, vilket förenar produkterna av projektionerna och gör det möjligt att vid en sökning på en specifik persons personnummer, få fram alla relaterade vänner, oberoende av om personen står som Pnr1 eller Pnr2 i just den instansen av relationen. Denna vyen är fördelaktig för informatörnissen då denne bör upprätthålla att alla barn endast gör inspelningar om de personer som de ingår i vänskapsrelationer med, denna tabellen möjliggör denna filtrering/matchning. Samt kan barn få en anpassad vy där de kan se vilka personer som de ingår i vänskapsrelationer med och på så sätt kan de notera vem de kan göra inspelningar om, en identiskt fast justerad vy som inte visar pnr hade kunna implementeras för dem i framtiden.

1.7.2 Specialization - 1 (VG)

Kodexempel 15

```
-- Specialisation 1
-- Snälla barnens hjälpsamhetsvärde

CREATE VIEW snallBarn_hjalpsamhet AS
SELECT
    barnPnr,
    barnFnamn,
    barnEnamn,
    COUNT(*) AS hjalpsamhetsvarde
FROM sysslorLogg
WHERE genomForande IN ('Klantigt', 'Lyckat')
GROUP BY barnPnr, barnFnamn, barnEnamn;
select * from snallBarn_hjalpsamhet;
```

Inom 'Kodexempel 15, skapas vyen ‘snallBarn_hjalpsamhet’ med syftet att räkna ut hjälpsamhetsvärdet för snälla barn, enligt följande direktiv ‘För snälla barn ska det gå att beräkna ett hjälpsamhetsvärde genom att räkna de sysslor som är klantigt genomförda eller bättre’ (Gustavsson & Bjurén, 2022). Eftersom sysslor endast görs av snälla barn kommer endast tabellen ‘sysslorLogg’ användas för den följande frågesatsen.

Först görs en projektion på barnets identifierare inom ‘sysslorLogg’, då dessa kommer representera vilket barn som hjälpsamhetsvärdet tillhör.

Därefter görs ytterligare en projektion med ‘COUNT(*)’ funktionen, vilket räknar hela rader inom tabellen sysslorLogg, och i WHERE klausulen specificeras vilka rader inkluderas i räkningen. I detta fallet räknas endast rader där attributet ‘genomForande’ har ett värde av ‘Klantigt' eller ‘Lyckat’ och resultatet per rad tilldelas kolumnnamnet: ‘hjalpsamhetsvarde’.

Slutligen grupperas resultatet efter barnens identifierare. Detta sammanfogar alla mulitpla instanser av barnet inom ‘sysslorLogg’ och sammanställer då resultaten av varje matchande tuppels ‘hjalpsamhetsvarde’. Vilket i sin tur ger varje barns totala, ackumulerade och summerade hjälpsamhetsvärde inom en rad.

Den målgrupp som kan dra nytta av denna viewen förklaras inte explicit enligt Gustavsson och Bjuréns (2022). Men kan antas vara exempelvis informatörer, handläggare och barn. T.ex handläggare och informatörer använda detta värdet inom deras avväganden och barnet kan få en överblick i hur hjälpsamma de är och kanske tillochmed i förhållande till andra barn, vilket skulle kunna ‘spelifiera’ aspekten i att utföra mest sysslor och vara mest hjälpsam.

1.8. Stored procedures

1.8.1 Procedur 1

Kodexempel 16

```
-- "(G) procedur1: enkel procedur, ta fram data när proceduren körs"
DELIMITER //

CREATE PROCEDURE hamta_inspelningar_Kvalitet (
IN Onskad_kvalitet_Inspelning tinyint
)
BEGIN
   select *
   from inspelning
   where kvalitetsVarde=Onskad_kvalitet_Inspelning;
END //

DELIMITER ;
call hamta_inspelningar_Kvalitet (10);
```

För proceduren inom ‘Kodexempel 16’ ryms en selektion som hämtar rader inom ‘inspelning’ tabellen som matchar användarens input.

Proceduren skapas genom ‘CREATE PROCEDURE’ uttrycker och fågeuttrycket avgränsas inom ‘BEGIN’ och ‘END’ kommandot. Hela proceduren omsluts sedan av en ‘DELIMITER’, vilket säkerställer att hela proceduren körs och inte bara fram tills det första uttryckets delimiter av ’ ;’. Proceduren har en parameter av ‘Onskad_kvalitet_Inspelning’ och den kommer användas i ‘WHERE’-klausulen för endast returnera de rader vilket är av samma värde som parametern. Exempelvis är parametern inom ‘Kodexempel 16’ siffran 10 och därmed hämtas de rader i ‘inspelning’ vilket har ett matchande värde i ‘kvalitetsVarde’-kolumnen.

Fördelen med denna procedur blir att rätt på ett kontrollerat sätt kunna hämta en önskad önskelistrad, utan extrern påverkan på koden vilket ekeveras då den är enkapsulerad i en procedur så vi vet vilken kod som körs. Proceduren är även fördelaktig för informatörer, då de kan få tillgång till proceduren för sökning av inspelningar med specifika värden för att exempelvis korrigera värden, samt kan de föra rapporter och statistik. Då en inspelning ‘betraktas som färdigbehandlad när inspelningen har tilldelats ett kvalitetsvärde’ (Gustavsson & Bjurén, 2022) kan proceduren användas för informatörerna när de söker upp vilka inspelningar som är obehandlade, dvs inget kvalitetsvärde.

1.8.2 Procedur 2

Kodexempel 17

```
--   procedur2: en procedur som tar emot en parameter och (tar fram data)/(tarbort
data)/(förändrar data) med hjälp av parametern, tex alla fakturor äldre än inmatat
datum och visar resultatet.

DELIMITER //

CREATE PROCEDURE hittaInspelningar (
IN BarnPersonnummer char(14),
BarnFornamn varchar (20),
BarnEternamn varchar (20)
)
BEGIN
    select
    tidsstampel as Skickades,
    barnPnr as Personnummer,
    barnFnamn as Fornamn,
    barnEnamn as Efternamn,
    -- exkluderar nisseid
    barn_om_Fnamn as Kompis_Fornamn,
    barn_om_Enamn as Kompis_Efternamn,
    beskrivning as Beskrivning,
    filNamn as Inspelning
    from inspelning
    where barnPnr = BarnPersonnummer and
                barnFnamn = BarnFornamn and
                barnEnamn = BarnEternamn;

END //

DELIMITER ;

CALL hittaInspelningar('00022345670211', 'Bertil', 'Bergström');
```

Inom koden i ‘Kodexempel 17’ skapas en procedur med namnet hittaInspelningar, vilket har i syfte att hitta inspelningar, via matchning av det inspelande barnets unika identifierare (inte subjektet dvs barnet det handlar om). Proceduren tar tre parametrar: ’IN BarnPersonnummer char(14), BarnFornamn varchar (20), BarnEternamn varchar (20)’ och inuti procedurens kropp görs en projektion av relevanta attribut för barnet om dess inspelning och dessa kolumner döps om till relevanta värden. Detta inkluderar tidstämpel som ‘skickades', barnets identifikation som personnummer, för och efternamn. informatörnissens ID projiceras inte, då endast barnen och information om inspelningen, är relevanta i svaret. Slutligen matchar projektionen till parametern för filtrering av de rader där de önskade barnet gjort en inspelning.

Proceduren kan användas informatörnissen eller barnet, ifall de vill åt en speciell persons inspelning. I barnets fall är det endast deras egna inspelningar vilket är av intresse, för exempelvis feedback om hur deras inspelningar är graderade. I informatörens fall kan det gälla ett rapportering, åtgärder eller statistikföring av ett visst barns inspelningar. De nämnda användarna kan då enkelt exekvera färdigbyggd kod med hjälp av en input, där de kan förvänta sig säkra och träffsäkra svar, istället för att behövs omdefiniera queryn varje gång per användare.

1.8.3 Procedur 3

Kodexempel 18

```
-- (VG) procedur3: samma som procedur2 men att även inkludera användandet av conditions
(if/else) och vid fel – ge ett egenskapat felmeddelande

-- (Antas att ett barn registrerar sina egna sysslor.)

DELIMITER //

CREATE PROCEDURE sysslaRegg (
IN BarnPersonnummer char(14),
BarnFornamn varchar (20),
BarnEternamn varchar (20),
sysslaNamn varchar (20),
datumGjort datetime,
nivaAv_utForande ENUM('Undermåligt', 'Klåparmässigt', 'Klantigt', 'Lyckat')
)

BEGIN

Declare finnsSyssla tinyInt;

Select count(*)
into finnsSyssla
from sysslorLogg
where
  barnPnr = BarnPersonnummer
  AND barnFnamn = BarnFornamn
  AND barnEnamn = BarnEternamn
  AND syssla = sysslaNamn
  AND sysslaTidDatumFardig = datumGjort
  AND genomForande = nivaAv_utForande;

IF (finnsSyssla = 0)

Then
     INSERT INTO sysslorLogg (barnPnr, barnFnamn, barnEnamn, syssla,
sysslaTidDatumFardig, genomForande)
     VALUES (BarnPersonnummer, BarnFornamn, BarnEternamn, sysslaNamn,
datumGjort, nivaAv_utForande);

else

    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Den angivna sysslan existerar redan.';

end if;

END //

DELIMITER ;

CALL sysslaRegg(
    '00011234560110', -- BarnPersonnummer
    'Anna',           -- BarnFornamn
    'Andersson',      -- BarnEternamn
    'Damma',   -- sysslaNamn (new task)
    '2024-09-12 14:30:00', -- datumGjort (new date-time)
    'Lyckat'          -- nivaAv_utForande
);
```

Koden i ‘Kodexempel 18’ deklarerar en procedure med namnet ‘sysslaRegg’. Proceduren har 6 parametrar: ‘BarnPersonnummer char(14), BarnFornamn varchar (20), BarnEternamn varchar (20), sysslaNamn varchar (20), datumGjort datetime, nivaAv_utForande ENUM('Undermåligt', 'Klåparmässigt', 'Klantigt', ‘Lyckat’)’. Inom dess kropp deklareras variabeln av ‘finnsSyssla’ vilket skall kontrollera ifall det finns en syssla, med matchande attribut till parametern. För varje rad där villkoret stämmer kommer count funktionen öka variabelns värde med 1 och om det inte finns någon syssla med de specificerade värden kommer finns ‘finnsSyssla’ vara noll. Finns ingen matchande syssla läggs den nya sysslan till i ‘sysslorLogg’ via ‘insert into’ och ‘values’. Om sysslan redan finns hanteras det av else -“scenariot”, vilket består av ett anpassat textuellt-felmeddelande med Signal SQL state ‘4500’.

Syftet med proceduren är att registrera nya sysslor in i ‘sysslorLogg’ tabellen. Via en villkorssats kommer en existerande syssla generera ett anpassat felmeddelande. Inputen i proceduren är alla kolumn-namn från tabellen ‘sysslorLogg’ och outputen blir anpassad utifrån om det specificerade barnets syssla redan utförts tidigare med samma värden: vid samma tid och med samma identifierare.

Denna procedur är utformad för undvikande av duplicering av sysslor i samband med registrering av nya sysslor, för upprätthållandet av dataintegritet. Snälla barn kommer använda proceduren för registrering av nya sysslor, vilket sedan kommer användas för kalkylering av deras hjälpsamhetsvärde. Det är därför kritiskt att de samlas in på ett korrekt sätt så att inget barn får en fördel genom exempelvis ‘slumpartade’ duplikat.

1.8.4 Procedur 4

Kodexempel 19

```
---- (VG) procedur 4: en procedur som flyttar en tupel från ena tabellen till den andra tabellen i din vertikala split
--
DELIMITER //

CREATE PROCEDURE levereraOnskelistRad(
IN Ej_betal_arTal year,
IN BarnPersonnummer char(14),
BarnFornamn varchar (20),
BarnEternamn varchar (20),
onskelistansRadNummer tinyint)
BEGIN

INSERT INTO levererade_onskeListRader(arTal, barnPnr, barnFnamn, barnEnamn, radNr, antal)
SELECT
arTal,
barnPnr,
barnFnamn,
barnEnamn,
radNr,
antal

FROM ejLevererad_onskeListRader

WHERE arTal=Ej_betal_arTal and barnPnr=BarnPersonnummer and barnFnamn=BarnFornamn and barnEnamn=BarnEternamn and radNr=onskelistansRadNummer;

DELETE FROM ejLevererad_onskeListRader
WHERE arTal=Ej_betal_arTal
and barnPnr=BarnPersonnummer
and barnFnamn=BarnFornamn
and barnEnamn=BarnEternamn
and radNr=onskelistansRadNummer;

END //

DELIMITER ;

CALL levereraOnskelistRad(2023, '00011234560110', 'Anna', 'Andersson', 1);
```

Inom ‘Kodexempel 19’ flyttas en tuppel från ‘ejLevererad_onskeListRader’ till ‘levererade_onskeListRader’. Proceduren tar emot parametrarna som tillsammans representerar den sammansatta nyckeln för en ‘ej levererad önskelistrad’. Parametrarna består av: “Ej_betal_arTal year, IN BarnPersonnummer char(14), BarnFornamn varchar (20), BarnEternamn varchar (20), onskelistansRadNummer tinyint”.

Inom procedurens kropps utförs en insert sats med de projicerade attributen från tabellen ‘ejLevererad_onskeListRader’. Attributen väljs ut baserat på ‘WHERE’ klausulen vilket filtrerar den projicerade raden baserat på ifall den matchar med parameterns sammansatta nyckel. Ifall raden existerar inom ‘ejLevererad_onskeListRader’, kopieras den över till ‘levererade_onskeListRader’.

Därefter raderas den ‘flyttade’ raden från ‘ejLevererad_onskeListRader’, genom att använda ‘DELETE FROM’, vilket specificerar en rad skall raderas från ‘ejLevererad_onskeListRader’ tabellen som matchar värdena av parametern.

Exempelvis flyttas ‘Anna Andersson’ och hennes önskelistrad till tabellen för levererade önskelistrader och återfinns därmed inte längre i ‘ej levererade’ önskelistraderna. Detta är viktigt för att kunna se vilka barn som fått sina önskningar.

Denna proceduren är menad för en användare med mycket behörighet och kommer vara väldigt användbar för hela produktionslinjen. Såsom en administratör, informatör eller handläggare, vilket hanterar leverans och registrering av önskelistor i systemet. Proceduren stödjer, indirekt, barn då det säkerställer en korrekt hantering av deras önskelistor och därmed kan de få sina klappar på ett mätbart och konsekvent sätt.

Kolla nästa sida för triggers.

1.9. Triggers

1.9.1 Trigger 1 - after delete (g)

Kodexempel 20

–Loggtabell för

```
CREATE TABLE leveransLogg (
     anv_namn varchar (40),
    operation char (6),
    arTal year,
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),
    radNr tinyint,
    antal tinyint,

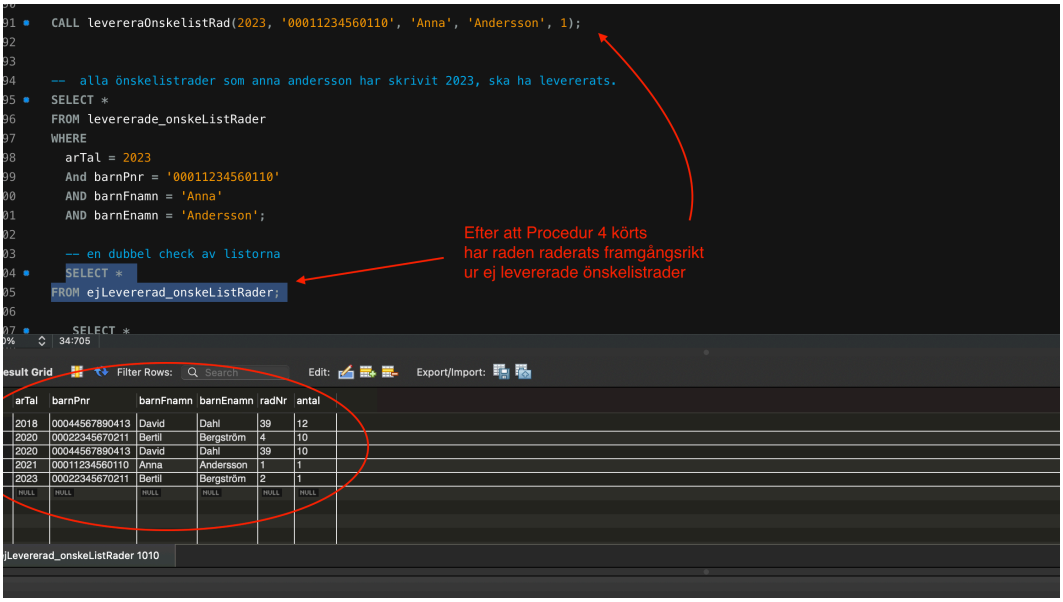
    Lev_datum_Logg datetime
) ENGINE=InnoDB;

-- TRIGGER 1 (g):

Delimiter //
CREATE TRIGGER logg_borttagning_ejlev after delete on ejLevererad_onskeListRader
for each row begin

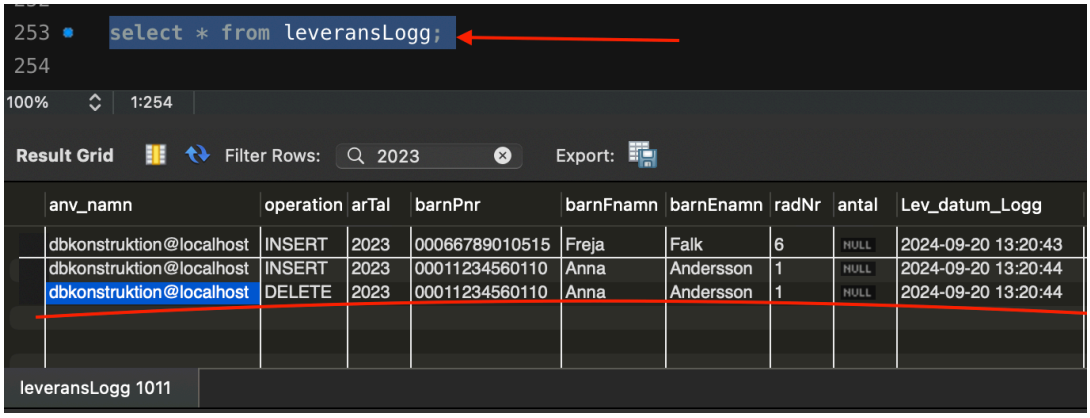
insert into leveransLogg (anv_namn, operation, arTal, barnPnr, barnFnamn, barnEnamn,
radNr, antal, Lev_datum_Logg)
values (user(), "DELETE", old.arTal, old.barnPnr, old.barnFnamn, old.barnEnamn,
old.radNr, antal, now());

END //
Delimiter ;
```



FIGUR 4 - PARAMETER BORTTAGEN UR 'EJLEVERERAD_ONSKELISTRADER'

FIGUR 5 - KONSLFÖNSTRET AV 'LEVERANSLOGG'



Inom ‘Kodexempel 20’ deklareras en trigger med namnet ‘logg_borttagning_ejlev’. Den skall utföras efter en delete operation utförs, korrekt, på tabellen ‘ejLevererad_onskeListRader’. När triggern går igång görs en insättning av den raderade raden in i ‘Lev_datum_Logg’, vars struktur går att notera ovanför ‘trigger 1’, den består av alla barnets identifierare, önskelistradens årtal, antal leksaker, radnummer, samt loggvärden av användarens namn, loggtid och typ av operation utförd. Insättningen sparar all information om den raderade raden, användaren vilket utförde raderings operationen och när raderingen av raden skedde.

Denna triggern skall arbeta synergistiskt med ‘procedur 4’ inom ‘Kodexempel 19’, genom att logga varje rad vilket flyttas från ej levererade önskelistrader till levererade önskelistrader, genom att kopiera över den, för att sedan radera raden från ej levererade önskelistrader. Detta levererar då den specificerade önskelistraden.

Längst ner i ‘Kodexempel 19’ kallas ‘Procedur 4’ på med en parameter av 'CALL levereraOnskelistRad(2023, '00011234560110', 'Anna', 'Andersson', 1);’. För testning av ifall triggern funkar kan vi följa denna förflyttning av ‘Anna Andersson’. Inom ‘Figur 4’ noteras ett anrop av procedur 4 vilket resulterar i en framgångsrik radering av ‘Anna Anderssons’ önskelistrad från 2023. Detta loggas framgångsrikt, vilket kan ses i ‘Figur 5’, där en ‘DELETE’ operation skedde på den flyttade tuppeln, dvs ‘Anna Andersson’ och denna är konsekvent till den givna exempel-parametern av ‘Procedur 4’.

Denna Triggern är användbar för barn eftersom att man kan hålla koll på vilka leveranser som flyttas, dvs levereras och vilka som tas bort från ej levererade tabellen. Detta gynnar barnen indirekt då det lättare kan skapas uppföljningar av felaktiga önskelistor/önskelistrader, samt att det dessutom bidrar det med transparens och spårbarhet genom att logga alla förändringar, vilket i sin tur skapar en tydlig historik över vad som har levererats och när. Därmed kan barnen få en översikt över vilka önskningar som har uppfyllts eller fortfarande väntar på leverans.

1.9.2 Trigger 2 (g)

Kodexempel 21

```
-- Loggtabell för trigger 2 (g):

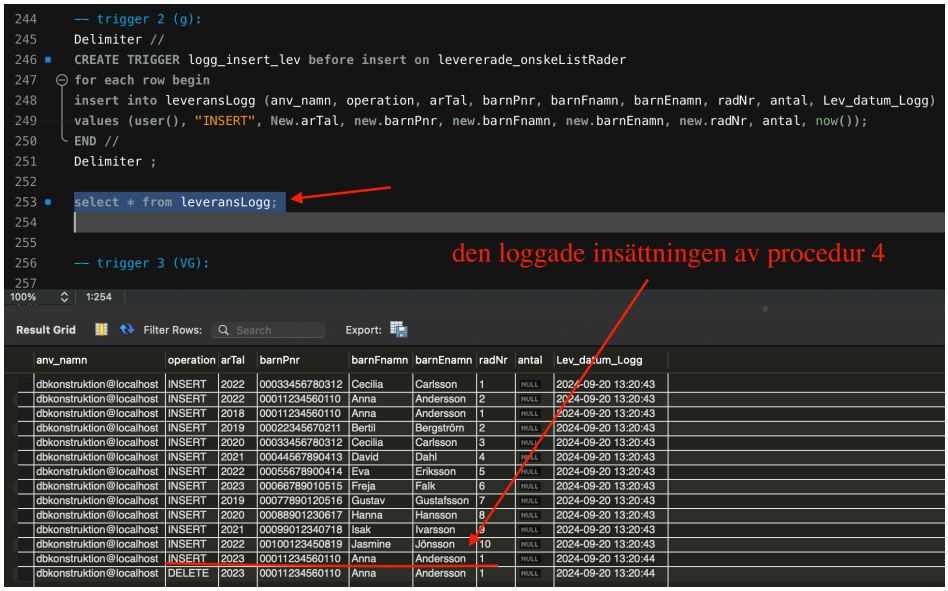
CREATE TABLE leveransLogg (
    anv_namn varchar (40),
    operation char (6),
    arTal year,
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20),
    radNr tinyint,
    antal tinyint,

    Lev_datum_Logg datetime
) ENGINE=InnoDB;


-- TRIGGER 2 (g):

Delimiter //
CREATE TRIGGER logg_insert_lev before insert on
levererade_onskeListRader
for each row begin
insert into leveransLogg (anv_namn, operation, arTal, barnPnr,
barnFnamn, barnEnamn, radNr, antal, Lev_datum_Logg)
values (user(), "INSERT", New.arTal, new.barnPnr, new.barnFnamn,
new.barnEnamn, new.radNr, antal, now());
END //
Delimiter ;
```

Inom ‘Kodexempel 21’ deklareras en trigger med namnet logg_insert_lev och syftet att logga innan varje ny insert till tabellen ‘levererade_onskeListRader’, inom loggtabellen ‘leveransLogg’. De loggade värdena inkluderar barnets identifierare, årtalet för önskelistraden, radnummer, antal leksaker, samt specifika loggvärden: användarnamn, loggtid och typ av operation utförd.



FIGUR 6 - KONSLFÖNSTRET AV ‘LEVERANSLOGG’

Triggern testas inom ‘Figur 6’ genom att göra kalla på ‘Procedur 4’ inom ‘Kodexempel 19’. Proceduren förflyttar en specificerad tuppel från tabellen ‘ejLevererad_onskeListRader’ till ‘levererade_onskeListRader’ med en ‘insert into’, vilket bör aktivera triggern. Det kan noteras längst ner i där ‘Procedur 4’ att den anropas med en parameter av 'CALL levereraOnskelistRad(2023, '00011234560110', 'Anna', 'Andersson', 1);’, Notera att denna återfinns inom figur 4 vid den markerade raden inom konsolfönstret. Där återfinns alla dess ursprungliga värden, samt tillämpade loggvärden. Därmed kan det konstateras att triggern funkar.

‘logg_insert_lev’ jobbar tillsammans med ‘Procedur 4’ och triggern ‘logg_borttagning_ejlev’ för att utföra en dubbel läggning av insättningar och raderingar av rader under processen då en önskelistrad levereras inom subsystemet. Denna dubbla loggningen säkerställer att barn kan lita på att deras önskelistor hanteras korrekt. Om något skulle gå fel, eller om ett misstag uppstår (som att en felaktig rad levereras eller tas bort), finns all nödvändig historik sparad. Vilket ger systemet möjlighet att snabbt identifiera och korrigera fel, vilket i slutändan gynnar barnen och ger dem insyn orderns status.

1.9.3 Trigger 3 (vg).

Kodexempel 22

```
-- Barn-tabell:

CREATE TABLE barn (
    Pnr char(14),
    Fnamn VARCHAR(20),
    Enamn VARCHAR(20),

    FodelseAR year NOT NULL,

    status_kommentar enum ('Aktiv', 'Saknar klassificering') default 'Aktiv',

    PRIMARY KEY (Pnr, Fnamn, Enamn),
     CHECK (Pnr RLIKE '^[0-9]{4}[0-9]{6}[0-9]{4}$') -- kollar format på barnPnr enl
uppgiftsbeskrivningen

) ENGINE=InnoDB;




-- Loggtabell för TRIGGER 3:

CREATE TABLE Barn_status_logg (
    barnPnr CHAR(14),
    barnFnamn VARCHAR(20),
    barnEnamn VARCHAR(20), -- barnens identifierare

    borttagenTid datetime,
    operation char (25),
    anv varchar (40) -- loggvärden
) ENGINE=InnoDB;




-- TRIGGER 3 (VG):

DELIMITER //

CREATE TRIGGER logg_borttagning_mindreSnalltBarn
before DELETE ON MindreSnalltBarn
FOR EACH ROW
BEGIN
    -- Logga borttagningen i en loggtabell (exempelvis snalltBarn_logg)

    INSERT INTO Barn_status_logg (
        barnPnr,
        barnFnamn,
        barnEnamn,
        borttagenTid,
        operation,
        anv
    )
    VALUES (
        OLD.barnPnr,
        OLD.barnFnamn,
        OLD.barnEnamn,
        NOW(),
        'DELETED FROM snalltBarn',
        user ()
    );

    -- Uppdatera barnets status i barn-tabellen

    UPDATE barn
    SET status_kommentar = 'Saknar klassificering'
    WHERE Pnr = OLD.barnPnr
      AND Fnamn = OLD.barnFnamn
      AND Enamn = OLD.barnEnamn;
END //END //

DELIMITER ;
```

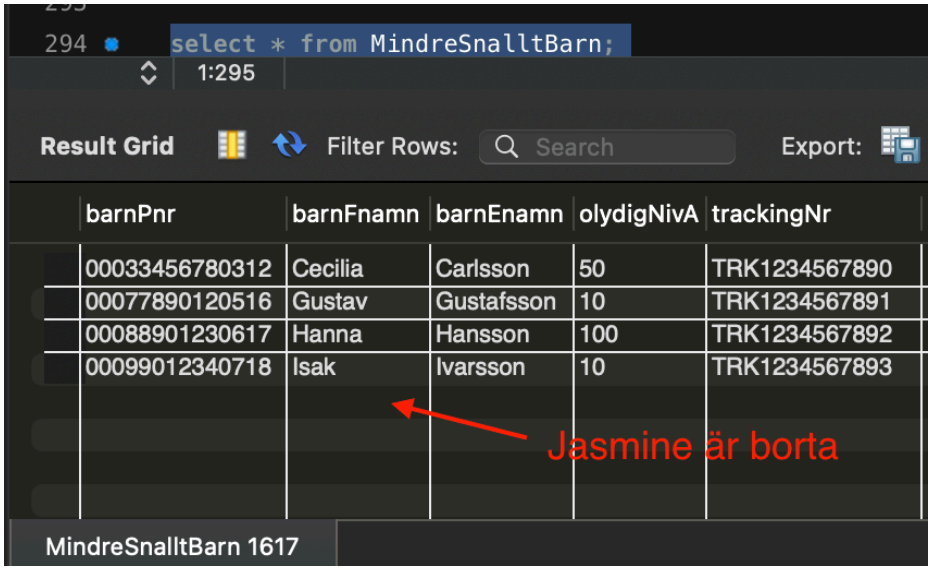
```
INSERT INTO MindreSnalltBarn (barnPnr, barnFnamn, barnEnamn, olydigNivA, trackingNr) VALUES
('00033456780312', 'Cecilia', 'Carlsson', 50, 'TRK1234567890'),

('00077890120516', 'Gustav', 'Gustafsson', 10, 'TRK1234567891'),
('00088901230617', 'Hanna', 'Hansson', 100, 'TRK1234567892'),
('00099012340718', 'Isak', 'Ivarsson', 10, 'TRK1234567893'),
('00100123450819', 'Jasmine', 'Jönsson', 50, 'TRK1234567894');

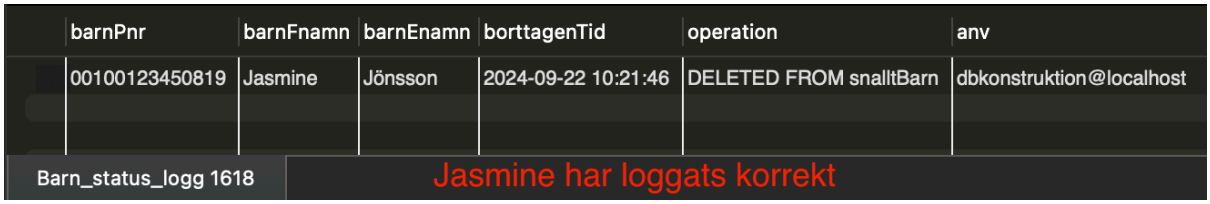
DELETE FROM MindreSnalltBarn
WHERE barnPnr = '00100123450819'
AND barnFnamn = 'Jasmine'
AND barnEnamn = 'Jönsson';
```

Jasmine tas bort från tabellen för mindre snälla barn

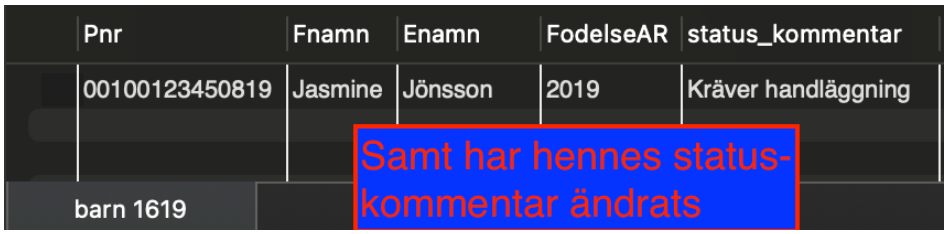
Figur 7 - Delete på 'MindreSnalltBarn' tabellen



Figur 8 - Delete funkade



Figur 9 - Korrekt loggning av Delete



Figur 10 - Korrekt uppdatering av 'status_kommentar' i 'barn'-tabellen

Inom ‘Kodexempel 22’ deklareras barntabellen, vilket kombinerar personnummer, förnamn, efternamn för identifiering. Samt lagras attributen av födelseår och status_kommentar. status_kommentar lagrar värdet statusen av barnet, då det antas att barnet endast kan vara aktivt om det är klassificerat inom dess underklasser: snällt eller mindre snällt.

Därefter deklareras loggtabellen: ‘Barn_status_logg’ vilket lagrar nödvändig metadata om barnet såsom dess primära nyckel, operationen, tid för operationen och vilken användare som utförde operationen gentemot barn. Tanken är att man kan implementera flera triggers för barn och logga olika operationer inom denna tabellen.

Därefter deklareras Trigger 3 ‘logg_borttagning_mindreSnalltBarn’. Triggern går igång efter en Delete operation på gjorts mot tabellen ‘MindreSnalltBarn’. För varje rad som tas bort från tabellen exekveras en ‘INSERT INTO’ tabellen ‘Barn_status_logg’ där borttagningarna lagras. Med de tidigare beskrivna värdena. Prefixen av ‘OLD’ används före insättningens kolumnnamn eftersom att datan skall raderas.

Det raderade barnets identifierare matchas med dess rad huvudtabellen: ‘barn’ och den motsvarande radens kolumn av ‘status_kommentar’ uppdateras från standardvärdet: ‘Aktiv’ till ‘Kräver handläggning’.

Inom ‘Figur 7, 8, 9 och 10’ går det att notera flödet av hur triggern funkar. Inom ‘Figur 7’ görs en DELETE operation på Jasmine Jönssons rad ur tabellen ‘MindreSnalltBarn’. Därefter kontrolleras borttagningen av Jasmine i samma tabell (‘Figur 8’). Inom ‘Figur 9’ återfinns Jasmines rad i loggtabellen och i ‘Figur 10’ observeras en korrekt uppdatering på Jasmines rad-kolumn av ‘status_kommentar’ från ‘Aktiv’ till ‘Kräver handläggning’.

Implementeringens struktur grundade sig i antagandet att barnen behöver kunna byta snällhetsstatus/ underkategori någon gång. Därmed implementerades en trigger där ev handläggare kommer kunna handlägga sådana flytningar via en transparent loggning av barn vilket saknar en underkategori. Statusen ska gå att filtrera på och barn som blivit snälla eller mindre snälla ska kunna byta underkategori på ett dokumenterat och transparent sätt inom systemet. Nästa steg i denna implementeringen skulle vara att göra en liknande trigger för borttagning av snälla barn samt en stored procedure för integreringen av barnets rad i den motsatta underkategori/flytten.

1.9.4 Trigger 4 (vg)

```
'Kodexempel 23'

-- Trigger 4 (VG):

DELIMITER //

CREATE TRIGGER satt_mp4_postfix_och_logga_inspelning
BEFORE INSERT ON inspelning
FOR EACH ROW
BEGIN
    -- Kolla om filnamnet inte slutar på '.mp4'
    if right (new.filNamn, 4) != '.mp4' then
        -- Lägg till '.mp4' till filnamnet
        set new.filNamn = CONCAT(new.filNamn, '.mp4');
    end IF;

    -- logga alla insättningar av inspelningar
    insert into inspelning_logg (
            anv_namn,
            operation,
            ins_dat_logg,

            inspelningens_tidsstampel,
            barnPnr,
            barnFnamn,
            barnEnamn,
            barn_om_Pnr,
            barn_om_Fnamn,
            barn_om_Enamn,

            NisseID,
            beskrivning, - - görs av barnet
            kvalitetsVarde,
            filNamn
    )
    values (
             USER(),
           'INSERT',
            NOW(),
            new.tidsstampel,
            NEW.barnPnr,
            NEW.barnFnamn,
            NEW.barnEnamn,
            NEW.barn_om_Pnr,
            NEW.barn_om_Fnamn,
            NEW.barn_om_Enamn,
            NEW.NisseID,
            NEW.beskrivning, -- görs av barnet
            NEW.kvalitetsVarde, -- loggas pga anv i uträkning
            NEW.filNamn
    );

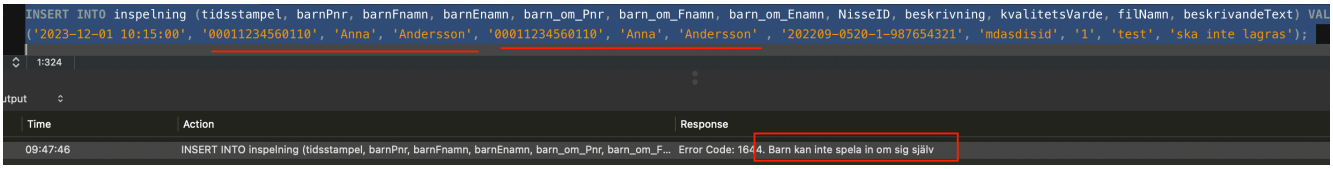
    -- ifall barnen har samma nyckel kan inte inspelning göras
   IF NEW.barnPnr = NEW.barn_om_Pnr
and NEW.barnFnamn=new.barn_om_Fnamn
and  NEW.barnEnamn=new.barn_om_Enamn
    THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Barn kan inte spela in om sig själv';
    END IF;
END //

DELIMITER ;
```

Inom Kodexempel 23 deklareras triggern: ‘satt_mp4_postfix_och_logga_inspelning’, med syftet att konkatenisera filnamnet med ’.mp4’ om filformatet inte har angetts vid insättningen samt att logga alla korrekta insättningar i ‘inspelning’-tabellen.

Inom triggerns kropp körs, för varje rad, en IF-sats där villkorssatsen kollar om filnamnet inte slutar på ’.mp4’. Om villkoret uppfylls, konkateniseras den insatta filNamn-kolumnens sträng med den avslutande strängen ’.mp4’, vilket anger dess filtyp. Om villkoret inte stämmer, konkateniseras strängen ej. Därefter loggas följande informationen in i loggtabellen ‘inspelning_Logg’ (‘Figur 11’): inspelningens tidstämpel, information om barnet som skapade inspelningen, subjektets personuppgifter, filnamnet, samt loggvärden för insättningsdatum och tid, användaren som utförde insättningen och den operation som genomfördes (’INSERT’). Samt barnets egen beskrivning av inspelningen.

I den nedre delen av triggerns kropp implementerades en IF-sats som kontrollerar om barnet som gör inspelningen har sig själv som subjekt. Detta görs genom att jämföra barnets och subjektets personuppgifter. Dvs: personnummer, förnamn och efternamn. Om villkoret uppfylls, det vill säga att barnets personuppgifter matchar subjektets, genererar triggern ett felmeddelande med texten "Barn kan inte spela in om sig själv" och utlöser en SQL-signal med SQLSTATE '45000'. Detta säkerställer att en inspelning där barnet spelar in om sig själv förhindras.



FIGUR 7- FELAKTIG INSÄTTNING TILL ‘INSPELNING’

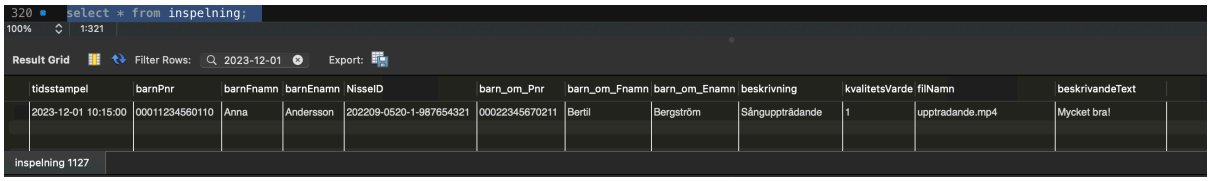


FIGUR 8- INGEN MOTSVARANDE RAD I ‘INSPELNING’

I ‘Figur 7‘ noteras en felaktig insättning till inspelningstabellen. Där barnets identifierare är identiskt till subjektets och därmed utlöses felmeddelandet inom konsolfönstret och inom ‘Figur 8’ kan vi se att ingen motsvarande rad med duplikat av ‘Anna’ existerar.

```
INSERT INTO inspelning (tidsstampel, barnPnr, barnFnamn, barnEnamn, barn_om_Pnr, barn_om_Fnamn, barn_om_Enamn, NisseID, beskrivning, kvalitetsVarde, filNamn, beskrivandeText) VALUES
('2023-12-01 10:15:00', '00011234560110', 'Anna', 'Andersson', '00022345670211', 'Bertil', 'Bergström' , '202209-0520-1-987654321', 'Sånguppträdande', '1', 'upptradande', 'Mycket bra!'),
```

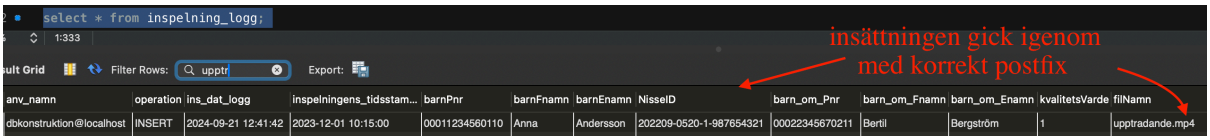
FIGUR 9 - EXEMPEL: GILTIG INSÄTTNING



FIGUR 10 - KONSOLFÖNSTER GILTIG INSÄTTNING

Enligt Gustavsson och Bjurén (2022) gör barnen inspelningar inom ett separat system, och det antas att informatören har tillräcklig behörighet och ansvar för att registrera nya inspelningar i subsystemet. Detta kan inkludera att lägga till filtypen i filnamnet. Eftersom alla system inte är kompatibla, antas det att det kan vara nödvändigt att tillägga filtypen för varje filnamn, för att filerna ska kännas igen och användas korrekt. Därmed görs antagandet att filerna endast kommer att vara av typen ‘mp4’ inom subsystemet.

Inom figur 9 och 10 går det att notera hur en korrekt insättning hanteras. ‘Figur 9’ visar hur insättningen har ett unikt barn och subjekt, samt har ‘filNamn’ kolumnen ingen filtyp vilket triggern åtgärdar inom ‘Figur 10’.



FIGUR 11 - GILTIG INSÄTTNING INSPELNINGSLOGGTABELL

Motsvarande rad loggas sedan också inom ‘Figur 11’, enligt tidigare beskrivning, med de unikt specificerade barnen och en korrekt postfix. Därmed funkar triggern.

1.10. Rights

```
Kodexempel 24
--
-- Rights
-- ------
-- --------

-- Skapar en testanvändare för barn gruppen

Create user 'test_barn'@'%' identified by 'Testlösenord';


-- test_barn får läsa, posta och uppdatera: ‘ejLevererad_onskeListRader'

Grant select, insert, update on a23guero.ejLevererad_onskeListRader to 'test_barn'@'%';


-- Ger test_barn rättigheter att läsa data från vyn barnVanner, där de kan se alla sina
vänner

Grant select on a23guero.barnVanner to ‘test_barn’@'%';



-- Ger test_barn tillgång till att hitta sina specifika inspelningar

Grant execute on procedure a23guero.hittaInspelningar to ‘test_barn'@'%';
```

Inom kodexempel 24 skapades användaren ‘test_barn'@'%' vilket identifieras med lösenordet ‘Testlösenord’. Notera att användaren kan få tillgång till databasen från valfri host/ip adress, via @‘%’. Användaren skall representera en instans av användargruppen ‘barn’, och vid senare iterationer av subsystemet kommer profilbaserad inloggning implementeras. Pga av säkerhetsskäl kommer inte inloggningsuppgifterna sättas till barnens personuppgifter, då dessa blir mer lätta att gissa än pseudonymer och egenpåkommna lösenord. Kan t.ex ett barns kompis dennes fullständiga namn och personnummer kan obehörig access fås, samtidigt som andra roller i systemet, av t.ex informatör och handläggaren har möjlighet att falskt kunna identifiera sig likt barn, de har tillgång till, i systemet. Då de enligt Gustavsson & Bjurén (2022) ska kunna se barnens i deras ‘vy’ av subsystemet där.

1.10.1 Tabell - ‘ejLevererad_onskeListRader’

Implementeringen av rättigheter till ‘ejLevererad_onskeListRader’ tabellen motiverades utifrån följande direktiv ‘Barnen ska kunna skapa nya önskelistor och ändra i existerande önskelistor. De ska även kunna skriva text eller ladda upp filer (endast namnet på filen hanteras i informationssystemet, själva uppladdning av filer hanteras i separat system)’ (Gustavsson & Bjurén, 2022).

Testanvändaren får tillgång till att:

* **SELECT**: Läsa data från tabellen

* **INSERT**: Lägga till nya poster i tabellen

* **UPDATE**: Uppdatera/ändra existerande poster i tabellen

Därmed kan de se, skapa, ändra önskelistor vilket ej har levererats och detta återspeglar därmed verkligheten. DELETE funktionen exkluderades, då den explicit inte tillfrågades per Gustavsson & Bjurén (2022), samt för att förhindra att barn tar bort sina rader av misstag. En potentiell lösning kan vara att barnen istället kan uppdatera ‘Antal’ kolumnen till noll och då kan en trigger implementeras för att nullifiera denna önskelistraden.

1.10.2 Vy - barnVanner

Implementeringen förhöll sig till följande direktiv: ’Ett barn kan även vara relaterat till flera olika barn. För varje sådan koppling mellan två barn lagras hur barnen är relaterade exv “syskon”, “kusin”, “dagis-kollega”’ (Gustavsson & Bjurén, 2022). Denna relation representeras av den tvåsidiga vyn ‘barnVanner’, men exkluderar vilken typ av relation det är. Då det kan implementeras vid senare iteration av systemet.

Testanvändaren får tillgång till att:

* **SELECT:** Läsa data från vyn och därmed kunna se vilka personer de ingår i en vänskapsrelation med.

Då vyn visar ‘vänskaper’ ur ett tvåsidigt perspektiv kan barnen på ett verklighetsbaserat och intuitivt sätt se vem de är vän med och om dem är vän med dem tillbaka. På det sättet kan barn göra korrekta och informerade inspelningar baserat på sina ‘vänskaper’.

Anledningen ‘test_barn’ inte får en INSERT av nya vänskaper på ‘barnVanner’ är för att det är en virtuell tabell vilket inte påverkar den ursprungstabellen: ’barn_vanskap’. Denna rättigheten kan, i framtida iterationen istället supplementeras via en ‘SELECT’ och en ‘INSERT’ mot ursprungstabellen ’barn_vanskap’. Då kan barnen rapportera in nya vänner, vilket sedan automatiskt kommer reflekteras på den virtuella tabellen av ‘barnVanner’.

1.10.3 Procedur - hittaInspelningar

Implementeringen baserades på följande direktiv: ‘Barnen ska få mata in person-information om sig själva i systemet samt läsa sina egna önskelistor eller de inspelningar/inlägg som de gjort om sina syskon’ (Gustavsson & Bjurén, 2022).

* **Execute: ‘**test_barn' får tillgång till exekvering av proceduren ‘hittaInspelningar’.

Rättigheten till ‘hittaInspelningar’ ger barnen möjlighet att söka upp inspelningar de har gjort, med parametern av deras egen identifierare/personuppgifter. Vilket uppfyller att de ska kunna 'läsa sina egna’ inspelningar/inlägg. Att ge dem tillgång till detta inom en procedur begränsar åtkomsten till de övriga barnens data och undviker obehörig manipulation av data via exempelvis sql injektions, då proceduren har en fördefinerad kropp och specificerade parametrar.

Referenser

Gustavsson, H. & Bjurén, J., 2022. *Databassystem: Databaskonstruktion Ht 2022 Inlämningsuppgift*. [Hämtad 8 september 2024].